



Università degli Studi dell'Aquila

Dipartimento di Ingegneria e Scienze dell'Informazione e Matematica

Corso di Laurea in Informatica

Tesi di Laurea Magistrale

Sintesi di comportamenti collaborativi per sistemi multi-robot

Relatore

Prof. Massimo Tivoli

Laureando

Giovanni Spaziani

Anno Accademico 2024-2025

"Si realizzano sempre le cose in cui credi realmente;
e il credere in una cosa la rende possibile."
- Frank Lloyd Wright

Indice

1 Introduzione **2**

2 Contesto e motivazioni **3**

2.1 Introduzione iniziale ai sistemi multi-robot 3

2.2 Architetture di coordinamento nei sistemi multi-robot 3

2.2.1 Architetture centralizzate 4

2.2.2 Architetture gerarchiche 4

2.2.3 Architetture ibride 4

2.2.4 Architetture decentralizzate (coordinamento distribuito) 5

2.2.5 Confronto tra le architetture di coordinamento 6

2.3 Scopo della tesi 8

2.3.1 Il problema principale del coordinamento distribuito 8

2.3.2 Importanza del problema 8

2.4 Approccio adottato 9

2.5 Introduzione al caso di studio 9

3 Stato dell'arte **11**

3.1 Obiettivo dello stato dell'arte e criteri di confronto 11

3.2 Coordinamento nei MRS e task allocation 12

3.3 Approcci Model-Driven e DSL per la programmazione di MRS 13

3.4 Workflow e BPMN per la modellazione di collaborazioni multi-robot . . 14

3.5 Approcci di verifica formale delle collaborazioni 15

3.6 Confronto e posizionamento del contributo 16

4 Background teorico **18**

4.1 Coreografie 18

4.1.1 Definizione di coreografia 18

4.1.2 Coreografie di servizi 18

4.2 Specifica di coreografie 19

4.2.1 Coreografia come specifica formale 19

4.2.2 Rappresentazione grafica della coreografia (BPMN 2.0) 21

4.2.2.1 Choreography diagrams e differenza con i collaboration diagrams . . . 21
4.2.2.2 Choreography task . . . 22
4.2.2.3 Principali elementi della notazione BPMN 2.0 . . . 22
4.2.3 Semantica della coreografia (stati e transizioni) . . . 24
4.2.4 Tipi di stati nella coreografia . . . 25
4.3 Ulteriori notazioni di specifica utili nel contesto dei MRS . . . 27
4.3.1 Modelli di comportamento e specifica dei compiti nei MRS . . . 27
4.3.2 Modelli di interazione e coordinamento tra robot . . . 28
4.3.3 Relazione tra specifica di missione e coreografie . . . 28
4.4 Realizability . . . 28
4.4.1 Cause di non realizzabilità . . . 29
4.4.2 Realizability nei MRS . . . 30
4.5 Problemi nella realizzazione delle coreografie . . . 31
4.5.1 Comunicazione sincrona e asincrona . . . 31
4.5.2 Deadlock e situazioni di stallo . . . 32
4.5.3 Inconsistenze di ordine e perdita di messaggi . . . 33
4.6 Strategie per affrontare la realizzabilità delle coreografie . . . 34
4.6.1 Verifica/analisi statica prima dell'esecuzione . . . 34
4.6.2 Repair della coreografia . . . 34
4.6.3 Enforcement a run-time . . . 38
4.7 Coordination Delegates . . . 38
4.7.1 Ruolo dei CDs nell'enforcement della coreografia . . . 39
4.7.2 Coordination Delegates e partecipanti black-box . . . 39
4.8 Le quattro fasi dell'approccio . . . 40
4.8.1 Proiezione . . . 40
4.8.2 Selezione . . . 42
4.8.3 Sintesi e concretizzazione . . . 43
4.8.4 Confronto tra le quattro fasi dell'approccio . . . 44

5 Metodo automatico per la collaborazione tra MRS 45
5.1 Introduzione del metodo automatico . . . 45
5.2 Input del metodo automatico . . . 46
5.3 Descrizione generale del metodo . . . 48
5.4 Compatibilità sintattica e semantica . . . 49
5.5 Ontologie e subsumption . . . 50
5.6 Testing comportamentale . . . 51
5.7 Analisi e disambiguazione dei possibili destinatari . . . 52
5.8 Applicazione del caso di studio . . . 54
5.8.1 Interazioni che partono dal Drone . . . 54

5.8.1.1 Prima interazione 54
5.8.1.2 Seconda interazione 54
5.8.2 Interazioni che partono dal Tagliatore 55
5.8.2.1 Prima interazione 55
5.8.2.2 Seconda interazione 55
5.8.3 Interazioni che partono dallo Scavatore 56
5.8.3.1 Prima interazione 56
5.8.4 Interazioni che partono dal Seminatore 56
5.8.4.1 Prima interazione 56
5.8.5 Esempio esteso $1 \rightarrow N$ 56
5.9 Legenda dei diagrammi BPMN 58
5.10 Choreography diagram globale 61
5.11 Diagrammi di proiezione 62
5.11.1 Proiezione del Drone 63
5.11.2 Proiezione del Tagliatore 63
5.11.3 Proiezione dello Scavatore 64
5.11.4 Proiezione del Seminatore 64
5.12 Diagrammi enhanced 65
5.12.1 Drone enhanced 65
5.12.2 Tagliatore enhanced 65
5.12.3 Scavatore enhanced 66
5.12.4 Seminatore enhanced 66
5.13 Unified collaboration diagram 67

6 Conclusioni e sviluppi futuri **69**

Elenco delle figure

2.1 Schema concettuale di un MRS con coordinamento distribuito. 5

3.1 Fasi del processo di sviluppo del framework FaMe. 14

4.1 Ciclo di vita di una coreografia e relazione con le orchestrazioni dei partecipanti. 19
4.2 Esempio della specifica di una coreografia. 20
4.3 Principali elementi notazione BPMN 2.0. 23
4.4 Esempio di una situazione di stallo causata da attesa reciproca. 33
4.5a Esempio della specifica di una coreografia. 37
4.5b Repair tramite relaxation (rimozione vincolo precedenza). 37
4.5c Repair tramite restriction (introduzione sincronizzazione). 38
4.6a Proiezione della coreografia sui partecipanti P_1 e P_2. 41
4.6b Proiezione della coreografia sui partecipanti P_3 e P_4. 42

5.1 Diagramma esplicativo del metodo. 46
5.2 Operazioni basilari dei robot 47
5.3 Riferimento ontologico. 51
5.4 Legenda dei diagrammi BPMN. 59
5.5 Choreography diagram globale. 61
5.6 Diagramma di projection del Drone. 63
5.7 Diagramma di projection del Tagliatore. 63
5.8 Diagramma di projection dello Scavatore. 64
5.9 Diagramma di projection del Seminatore. 64
5.10 Diagramma enhanced del Drone. 65
5.11 Diagramma enhanced del Tagliatore. 66
5.12 Diagramma enhanced dello Scavatore. 66
5.13 Diagramma enhanced del Seminatore. 67
5.14 Unified collaboration diagram. 67

Elenco delle tabelle

2.1 Vantaggi e svantaggi delle architetture decentralizzate. 6
2.2 Confronto tra le architetture di coordinamento. 7
2.3 Robot coinvolti nel caso di studio. 10

3.1 Criteri adottati per il confronto. 11
3.2 Analisi degli approcci MRTA. 12
3.3 Analisi degli approcci Model-Driven e DSL. 13
3.4 Analisi dell'approccio FaMe. 15
3.5 Analisi degli approcci di verifica formale. 16
3.6 Confronto tra gli approcci. 17

4.1 Elementi della specifica di una coreografia. 20
4.2 Elementi della LTS. 24
4.3 Cardinalità delle transizioni. 25
4.4 Plain state. 26
4.5 Branch state. 26
4.6 Fork state. 26
4.7 Join state. 27
4.8 Equivalenza linguistica. 29
4.9 Well formedness. 30
4.10 Confronto tra comunicazione sincrona e asincrona. 32
4.11 Confronto tra verifica/analisi statica e repair della coreografia. 35
4.12 Confronto tra relaxation e restriction. 35
4.13 Confronto tra le quattro fasi dell'approccio. 44

5.1 Esempio di messaggi scambiati nel caso di studio esteso $1 \rightarrow N$. 57

Capitolo 1

Introduzione

Nell'ambito della robotica, i sistemi multi-robot (Multi-Robot Systems, MRS) consentono a più robot autonomi di operare nello stesso ambiente e cooperare per il raggiungimento di un obiettivo condiviso.
Il presente elaborato dunque affronta il problema del coordinamento distribuito nei MRS, in cui il comportamento globale emerge grazie alle interazioni dei robot, dotati di capacità decisionali e operative locali. In assenza di adeguate garanzie di coordinamento, asincronia e autonomia dei robot possono compromettere la missione; è necessario quindi un approccio che assicuri coerenza globale mantenendo l'esecuzione locale e distribuita.
Dall'analisi dello stato dell'arte descritta nel Capitolo 2, si è notato che le diverse soluzioni esistenti non affrontano in modo esplicito il problema della sintesi automatica di questi protocolli partendo da una specifica globale del comportamento desiderato.
In questo contesto, il presente lavoro propone un approccio che adatta ai MRS i concetti di coreografie di servizi e di enforcement distribuito tramite i Coordination Delegates [3]. A partire da una descrizione globale della missione, vengono derivati comportamenti locali e vincoli di interazione che ne garantiscono la corretta realizzazione distribuita. In questo modo, il coordinamento nasce grazie alle interazioni che assicurano la coerenza rispetto alla visione globale del sistema.
Al fine di concretizzare i concetti introdotti, l'approccio descritto in questa tesi viene applicato ad un caso di studio basato su una flotta di robot che cooperano ed evidenziano la gestione distribuita delle dipendenze tra attività e punti di sincronizzazione. I risultati mostrano la fattibilità dell'approccio e le sue potenzialità per la progettazione di comportamenti collaborativi.
Il presente elaborato è strutturato come segue: Capitolo 2 introduce il contesto dei MRS e le problematiche legate al coordinamento tra entità autonome; Capitolo 3 presenta un'analisi dello stato dell'arte; Capitolo 4 fornisce il background teorico relativo alle coreografie, specifiche, criticità e meccanismi di enforcement; Capitolo 5 presenta il metodo proposto con relativa applicazione al caso di studio. Infine, Capitolo 6 termina l'elaborato con alcune considerazioni e possibili sviluppi futuri.

Capitolo 2

Contesto e motivazioni

2.1 Introduzione iniziale ai sistemi multi-robot

I sistemi multi-robot (*Multi-Robot Systems*, *MRS*) sono meccanismi formati da diversi partecipanti, operanti all'interno di uno spazio comune, con lo scopo di svolgere e portare a termine una missione condivisa [8]. Ogni robot che fa parte di questa missione percepisce l'ambiente, prende decisioni locali ed esegue le proprie azioni, contribuendo così al corretto comportamento del sistema. Quest'ultimo, quindi, è il risultato di interazione, cooperazione, coordinazione e comunicazione tra i robot coinvolti nella missione. A tal proposito, possiamo distinguere tra [8]:

- **MRS con orchestratore**, dove un singolo robot governa e controlla il comportamento dei restanti partecipanti alla missione.
- **MRS con coordinamento distribuito**, dove ogni singolo robot agisce in modo autonomo, confrontandosi con gli altri partecipanti per portare a termine la missione.

In questo lavoro di tesi, verrà preso in considerazione il caso di MRS con coordinamento distribuito, rappresentanti il contesto di riferimento del metodo automatico che verrà successivamente esplicato.

2.2 Architetture di coordinamento nei sistemi multi-robot

Nei MRS, un aspetto fondamentale del coordinamento riguarda la struttura del controllo e della presa di decisione (vale a dire il modo in cui le responsabilità della pianificazione e del controllo vengono distribuite tra i diversi robot che compongono il sistema).
Le principali architetture di coordinamento per i sistemi multi-robot sono [24]:

1. Centralizzate.
2. Gerarchiche.

3. Ibride.

4. Decentralizzate.

2.2.1 Architetture centralizzate

In questo tipo di architetture, il coordinamento del sistema multi-robot viene affidato ad una singola entità centrale che ha una visione globale di tutto lo stato del sistema, che si occupa di prendere tutte le decisioni che riguardano la pianificazione e il controllo delle azioni dei robot che fanno parte della missione (che hanno quindi poca autonomia decisionale e che fanno affidamento soltanto alle istruzioni che ricevono) [24].
Tuttavia, questo tipo di architettura presenta diversi svantaggi [27]:

- Risulta efficace soltanto per un numero molto limitato di robot.
- Se intorno all'ambiente dovessero esserci cambiamenti o problemi di comunicazione, questo tipo di architettura presenterebbe problemi di robustezza, poiché il coordinamento dipende soltanto dal nodo centrale.
- Nel caso in cui quest'ultimo dovesse riscontrare dei malfunzionamenti, necessiterebbe immediatamente di una sostituzione da parte di un nuovo agente (che si deve far trovare subito pronto) altrimenti l'intero team verrebbe disabilitato.

2.2.2 Architetture gerarchiche

Come lascia intendere il nome, in questo tipo di architetture, il coordinamento del MRS è organizzato secondo una struttura a più livelli, in cui i nodi superiori controllano quelli immediatamente inferiori, questi ultimi controllano quelli subito dopo, e così via, fino ad arrivare agli ultimi robot in gerarchia, che eseguono soltanto il compito che è stato loro assegnato [24].
Questo tipo di architettura permette di avere una migliore scalabilità rispetto a quelle presentate in 2.2.1; tuttavia, questo tipo di architettura ha un importante criticità [24]: i robot che nell'albero gerarchico si trovano ai livelli più alti, se dovessero presentare dei guasti, comprometterebbero il funzionamento di tutto il sistema, rendendo così difficile il totale recupero di esso.

2.2.3 Architetture ibride

Nel campo dei MRS, queste architetture integrano meccanismi di controllo locale con un livello di controllo che opera ad un livello di astrazione superiore rispetto al controllo locale, avendo come obiettivo quello di ottenere sia la robustezza dell'intero sistema sia la possibilità di influenzare il comportamento dell'intero team tramite strategie globali, piani o obiettivi [24].

2.2.4 Architetture decentralizzate (coordinamento distribuito)

Dopo l'introduzione dei sistemi multi-robot (Sezione 2.1) e delle prime tre architetture di coordinamento (Sezioni 2.2.1, 2.2.2 e 2.2.3), questa sezione approfondisce l'architettura decentralizzata, dove il coordinamento distribuito emerge dalle varie interazioni tra i robot.
Le architetture di questo tipo rappresentano l'approccio frequentemente utilizzato nei MRS; in tali architetture, ogni robot prende decisioni basandosi sulle informazioni locali relative alla propria situazione. Quindi si può affermare che non esiste un robot responsabile del controllo di tutti gli altri [24].

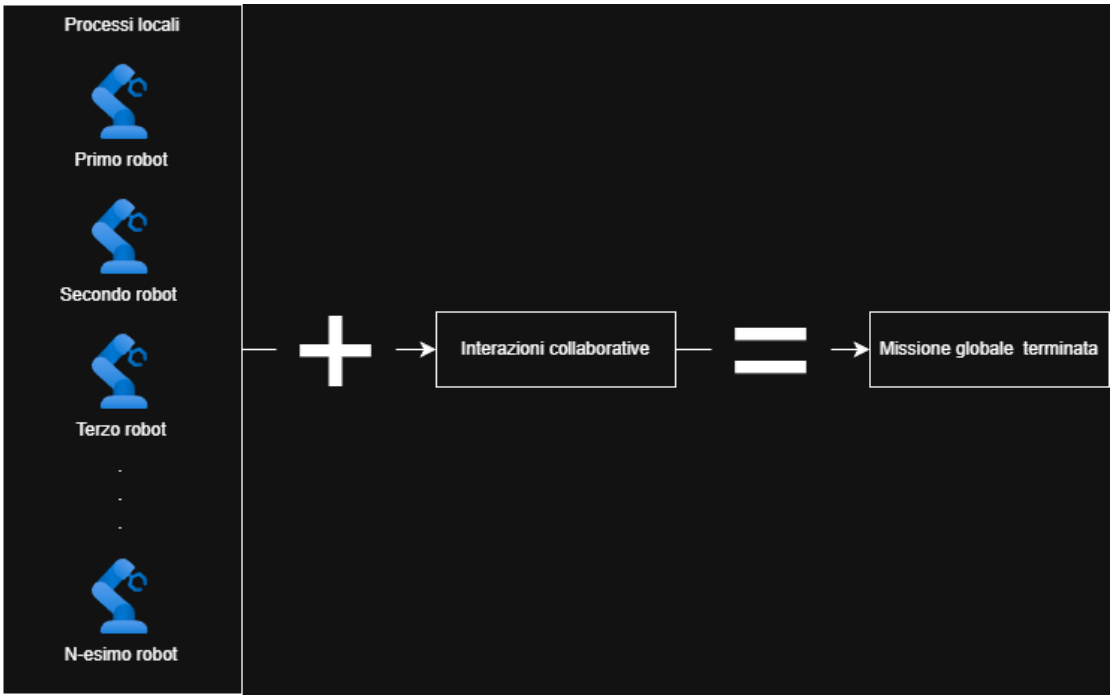


Figura 2.1: Schema concettuale di un MRS con coordinamento distribuito.

La Figura 2.1 mostra in modo semplificato il funzionamento di un MRS con coordinamento distribuito, dove la missione globale viene portata a termine attraverso l'integrazione dei processi locali dei singoli robot e delle loro interazioni collaborative.
Anche l'architettura decentralizzata presenta diversi vantaggi e criticità [24, 6], come discusso in Tabella 2.1:

Vantaggi	Svantaggi
Elevata robustezza ai guasti: i singoli robot non controllano e non sono responsabili degli altri partecipanti	Utilizzare robot multipli comporta un aumento dei costi
In questo tipo di architettura è possibile svolgere compiti che non potrebbero essere eseguiti da un singolo robot	I meccanismi utilizzati per coordinare i robot possono essere difficili da realizzare
Più robot agiscono allo stesso tempo anche in differenti luoghi distanti tra loro	In alcuni casi può accadere che le azioni di alcuni robot si sovrappongano, con conseguente perdita di prestazione
—	Se lo scopo della missione cambiasse, potrebbe risultare complesso cambiare il comportamento di ogni singolo robot
—	Gli obiettivi globali di tutto il team di robot, vale a dire la missione complessiva, devono essere tradotti in regole di comportamento locali per ciascun robot, e questa "traduzione" non è semplice

Tabella 2.1: Vantaggi e svantaggi delle architetture decentralizzate.

2.2.5 Confronto tra le architetture di coordinamento

Le principali caratteristiche delle architetture di coordinamento analizzate nelle Sezioni 2.2.1, 2.2.2, 2.2.3 e 2.2.4 vengono riassunte in Tabella 2.2, con lo scopo di facilitarne il contronto.

Architettura centralizzata		Architettura gerarchica		Architettura ibrida		Architettura decentralizzata	
Vantaggi	**Svantaggi**	**Vantaggi**	**Svantaggi**	**Vantaggi**	**Svantaggi**	**Vantaggi**	**Svantaggi**
Visione globale o completa dello stato del sistema	Efficace solo per un numero limitato di robot	Migliore scalabilità rispetto alla centralizzata	Criticità dei nodi ai livelli superiori	Integrazione di controllo locale e controllo di livello superiore	Maggiore complessità del sistema di controllo	Elevata robustezza ai guasti	Aumento dei costi dovuto all'uso di più robot
Pianificazione e controllo centralizzati delle azioni	Problemi di robustezza in presenza di cambiamenti o problemi di comunicazione	Struttura di controllo organizzata su più livelli	Guasti ai livelli alti compromettono il funzionamento dell'intero sistema	Maggiore robustezza complessiva del sistema	Necessità di integrare meccanismi eterogenei (locale + globale)	Nessun robot responsabile del controllo degli altri	Complessità dei meccanismi di coordinamento
Coerenza decisionale del team	Dipendenza dal nodo centrale	I robot ai livelli inferiori eseguono compiti assegnati, riducendo il carico decisionale	Recupero del sistema difficile in caso di guasti nella parte alta della gerarchia	Possibilità di influenzare il comportamento dell'intero team tramite strategie globali, piani o obiettivi	—	Possibilità di svolgere compiti non eseguibili da un singolo robot	Possibile sovrapposizione delle azioni con perdita di prestazioni
—	Un possibile guasto al nodo centrale disabiliterebbe l'intero team	—	—	—	—	Più robot possono agire simultaneamente in luoghi diversi	Difficoltà nel modificare il comportamento dei singoli robot se cambia lo scopo della missione
—	—	—	—	—	—	—	Difficoltà nel tradurre gli obiettivi globali in regole di comportamento locali

Tabella 2.2: Confronto tra le architetture di coordinamento.

Alla luce del confronto tra le diverse architetture di coordinamento, emerge come l'architettura decentralizzata rappresenti la soluzione più adatta per i MRS considerati in questo lavoro di tesi. In particolare, l'assenza di un'unità che funge da controllore, unita alla maggiore robustezza ai guasti e alla capacità di operare in contesti distribuiti e dinamici, risponde in modo efficare ai requisiti tipici dei MRS oggetto di studio. Per tali ragioni, il coordinamento distribuito costituisce il riferimento architetturale alla base del metodo proposto e che verrà analizzato nei capitoli successivi.

2.3 Scopo della tesi

2.3.1 Il problema principale del coordinamento distribuito

I sistemi multi-robot richiedono che più unità autonome cooperino per portare a termine una missione globale, senza utilizzare quindi un controllore centrale. Il nodo critico è rappresentato dal coordinamento distribuito, quindi garantire che tutti i robot che fanno parte di una missione facciano le seguenti cose:

- Eseguano correttamente le azioni nell'ordine previsto.
- Prendano decisioni coerenti in presenza di rami alternativi.
- Si sincronizzino correttamente nei punti di incontro della missione.

In assenza di meccanismi di coordinamento potrebbero emergere diversi problemi [27]:

- Divergenze nell'esecuzioni di rami decisionaili.
- Join prematuri o mancanti.
- Deadlock.
- Inefficienze dovute a possibile asincronia e limiti di comunicazione tra i robot.

Questi problemi possono compromettere l'esecuzione dell'intera missione, rendendo così complessa la gestione e la progettazione di comportamenti collaborativi in contesti distribuiti.

2.3.2 Importanza del problema

Il problema descritto in 2.3.1 è di grande rilevanza, dal momento in cui i MRS trovano applicazione in vari scenari complessi, che richiedono affidabilità, scalabilità e flessibilità. Gli approcci basati su un'unica entità di controllo, come descritto, mostrano diversi limiti significativi [24]; lo scopo, quindi, è quello di garantire un coordinamento corretto e totalmente distribuito, per consentire ai robot di portare a termine le missioni rispettando rispettando le specifiche anche in presenza di asincronia, limiti sulla comunicazione e variazioni dell'ambiente.

2.4 Approccio adottato

Per affrontare questo problema, questo lavoro di tesi propone di adattare al dominio multi-robot concetti già esistenti e consolidati nel contesto delle coreografie di servizi. In tale ambito, infatti, si è studiato come coordinare entità distribuite senza un orchestratore centrale, tramite l'utilizzo di specifiche globali e di meccanismi di enforcement distribuito, come i Coordination Delegates (che vedremo nel Capitolo 3), ovvero entità locali incaricate di garantire il rispetto della specifica globale attraverso sincronizzazioni distribuite tra i partecipanti [3].
L'idea alla base dell'approccio adottato in questo lavoro di tesi consiste nel trasferimento di questi concetti nel contesto dei MRS. In questo modo, il coordinamento non viene diretto da un'entità centrale, ma emerge dalle interazioni tra i comportamenti locali dei singoli robot, derivati dalla specifica globale.
L'approccio prevede quindi una pipeline concettuale che parte dalla definizione di una specifica globale del comportamento collaborativo, e prosegue con la derivazione di comportamenti locali corretti per ciascun robot, insieme a tutti i vincoli necessari per il corretto rispetto della specifica. Tale impostazione permette di mantenere una visione globale di tutto il comportamento del sistema, pur operando in un contesto completamente distribuito.
La novità del lavoro sta nell'introduzione di un approccio basato su una specifica globale del comportamento collaborativo per la sintesi del coordinamento distribuito nei MRS, a differenza degli approcci tradizionali che si concentrano prevalentemente su modelli di controllo locali o su architetture di coordinamento predefinite.

2.5 Introduzione al caso di studio

Al fine di concretizzare i concetti e le scelte introdotte in questo primo capitolo, nel seguito della tesi verrà considerato un caso di studio basato su una flotta di robot che coopera per l'esecuzione di una missione comune.
La Tabella 2.3 mostra da quali robot è formato il sistema preso in considerazione:

Robot	Compito svolto
Drone	Esplora l'area di interesse e individua le zone che richiedono intervento, fornendo informazioni al robot successivo
Tagliatore	Esegue il taglio delle erbacce nelle aree segnalate dal Drone, preparando il terreno per le operazioni successive
Scavatore	Prepara il terreno scavando nelle zone trattate dal Tagliatore, rendendo possibile la fase di semina
Seminatore	Effettua la semina nelle aree precedentemente preparate, completando la missione collaborativa

Tabella 2.3: Robot coinvolti nel caso di studio.

La missione considerata quindi prevede l'esecuzione di una serie di attività in cui ogni robot opera solo al termine delle fasi precedenti. Questo scenario è caratterizzato dall'assenza di un controllore centrale e dalla necessità di un coordinamento distribuito tra i vari robot, in grado di garantire il rispetto delle varie dipendenze tra i compiti.
Il caso di studio verrà poi utilizzato per illustrare come una specifica globale possa essere impiegata per derivare meccanismi di coordinamento distribuito nei MRS.

Capitolo 3

Stato dell'arte

3.1 Obiettivo dello stato dell'arte e criteri di confronto

Lo studio di coordinamento nell'ambito dei MRS ha prodotto nel tempo una vasta gamma di approcci, che si differenziano per modello architetturale, livello di distribuzione, meccanismi decisionali e strumenti di modellazione; tuttavia, essi non affrontano sempre il problema di sintesi dei protocolli collaborativi distribuiti avendo come punto di partenza la specifica globale [13].
Questo capitolo si occupa di analizzare gli approcci più pertinenti rispetto al problema affrontato in questo lavoro di tesi. L'analisi quindi si concentrerà su (Tabella 3.1):

Criterio	Descrizione
Nozione di coordinamento	Modalità con cui viene interpretato il coordinamento
Tipo di specifica adottata	Punto di vista con cui viene formalizzata la collaborazione
Meccanismo di realizzazione	Modalità di traduzione del modello in esecuzione
Distribuzione dell'esecuzione	Architettura con cui l'esecuzione è organizzata (centralizzata, gerarchica o distribuita)
Garanzie di correttezza	Operative/Verifica ex-post/Correttezza per costruzione

Tabella 3.1: Criteri adottati per il confronto.

Sulla base di questi criteri, l'analisi partirà dagli approcci classici, per poi considerare soluzioni model-driven e basate su workflow per la modellazione di missioni collaborative, fino ad arrivare agli approcci di verifica formale delle collaborazioni.

3.2 Coordinamento nei MRS e task allocation

Una delle problematiche maggiormente analizzate nell'ambito del coordinamento dei MRS è l'assegnazione dei compiti, dal momento in cui si deve determinare quale attività debba essere svolta da un determinato robot con lo scopo di raggiungere un obiettivo comune. Questa problematica è nota come Multi-Robot Task Allocation (MRTA), la cui formalizzazione e analisi hanno evidenziato le principali dimensioni strutturali e le diverse configurazioni variabili in base alla capacità dei robot di eseguire uno o più task, al numero di robot necessari per ciascun compito e all'orizzonte temporale dell'assegnazione, istantaneo oppure esteso nel tempo [20].
Le distinzioni appena elencate mettono in evidenza come il coordinamento venga modellato come problema di assegnazione dei compiti, senza l'introduzione di una rappresentazione globale esplicita delle interazioni tra robot.
Dentro la MRTA un approccio adottato è quello auction/market based, dove i task, le risorse ed i ruoli vengono assegnati tramite stime di costo ed utility, ed una fase finale assegna i task ai robot ritenuti migliori [18]; tuttavia, in presenza di dipendenze, sincronizzazioni o vincoli, la sola allocazione non basta a garantire la correttezza della distribuzione.
Nonostante questo tipo di approccio sia ritenuto un punto di riferimento per il coordinamento distribuito, non affronta il problema di definizione e sintesi di un protocollo globale di interazione tra i componenti del sistema.
In riferimento ai criteri definiti nella Tabella 3.1, l'approccio appena descritto viene analizzato nella Tabella 3.2.

Criterio	Descrizione
Nozione di coordinamento	Problema di allocazione e pianificazione dei compiti
Tipo di specifica adottata	Rappresentazione basata su task e assegnazioni
Meccanismo di realizzazione	Realizzano il comportamento tramite algoritmi decisionali
Distribuzione dell'esecuzione	Possono prevedere architetture distribute o ibride, spesso con meccanismi di coordinamento con orchestratore centrale
Garanzie di correttezza	Non forniscono garanzia sulla correttezza del protocollo e sulle proprietà di correttezza per costruzione

Tabella 3.2: Analisi degli approcci MRTA.

3.3 Approcci Model-Driven e DSL per la programmazione di MRS

L'ingegneria del software per la robotica nel tempo ha introdotto dei livelli di astrazione più elevati rispetto alla classica programmazione, favorendo così la generazione di artefatti eseguibili partendo dai modelli; in questo modo, grazie a questi principi provenienti dalla Model-Driven Engineering (MDE) è possibile separare la specifica del comportamento dalla relativa implementazione, migliorando la riusabilità, la possibilità di analisi del sistema e la manutenibilità [14].
Nel contesto dei MRS questi principi hanno trovato applicazione per la progettazione di missioni collaborative tramite Domain-Specific Languages (DSL). Un esempio di questo approccio è costituito da una famiglia di linguaggi utilizzati per la specifica di missioni civili multi-robot, organizzata in tre livelli: descrizione ad alto livello della missione, modellazione dei comportamenti dei robot, ed infine descrizione delle azioni eseguibili dai robot [17]. Il processo prevede trasformazioni da modello a modello e da modello a codice, in modo da ottenere modelli eseguibili a partire dalla specifica della missione. L'architettura può prevedere il coordinamento distribuito tra robot; tuttavia la pianificazione e la generazione dei comportamenti sono affidate ad un'infrastruttura che si occupa di gestire le trasformazioni dei modelli, l'orchestrazione delle attività e la comunicazione con i robot [17].
Questo approccio appena descritto viene analizzato nella Tabella 3.3 in base ai criteri definiti nella Sezione 3.1.

Criterio	Descrizione
Nozione di coordinamento	Modellazione e orchestrazione di task all'interno di una missione globale
Tipo di specifica adottata	Specifica dichiarativa multilivello
Meccanismo di realizzazione	Trasformazioni da modello a modello e da modello a codice
Distribuzione dell'esecuzione	Esecuzione dei robot distribuita, con pianificazione e coordinamento centralizzati
Garanzie di correttezza	Assenza di meccanismi corretti per costruzione e di garanzie formali integrate nell'architettura

Tabella 3.3: Analisi degli approcci Model-Driven e DSL.

3.4 Workflow e BPMN per la modellazione di collaborazioni multi-robot

Un approccio differente della modellazione delle collaborazioni nei MRS consiste nell'utilizzo di linguaggi di workflow standardizzati, in particolare del linguaggio BPMN (acronimo di Business Process Model and Notation). Quest'ultimo viene utilizzato per la descrizione esplicita delle interazioni tra i partecipanti della missione.
Il framework FaMe [13] propone un approccio del genere tramite l'utilizzo di un diagramma di collaborazione BPMN per la rappresentazione dello scenario. Inizia con la modellazione astratta sia del flusso delle operazioni che di quello delle interazioni e successivamente fornisce le linee guida per l'utilizzo degli elementi BPMN che stabiliscono un collegamento tra costrutti del linguaggio e concetti robotici.
Il suo processo di sviluppo è suddiviso in tre fasi (Figura 3.1) [13].

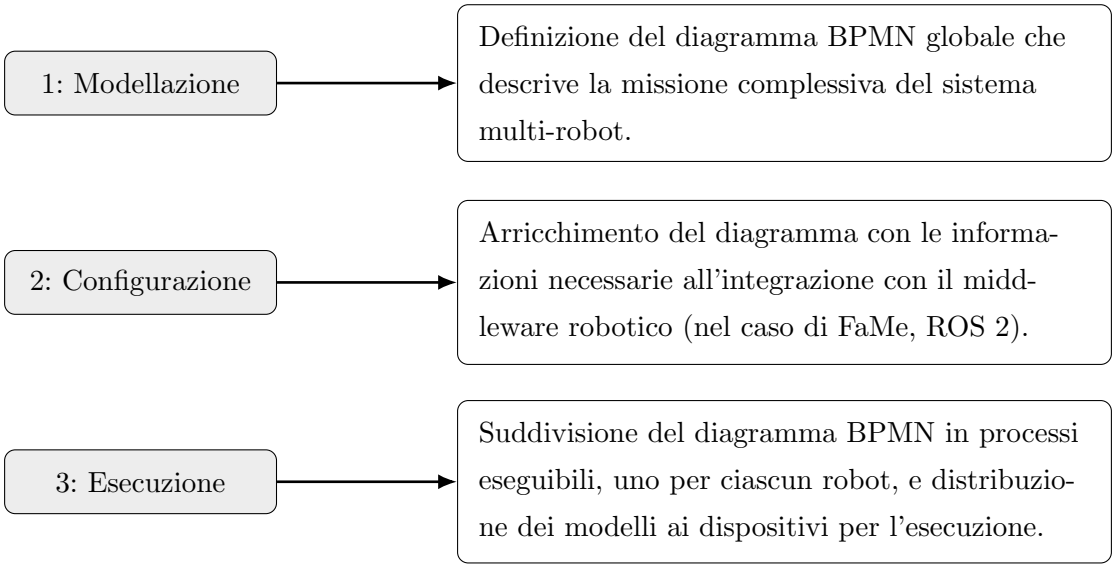


Figura 3.1: Fasi del processo di sviluppo del framework FaMe.

Nell'architettura finale ogni robot ha un motore di esecuzione BPMN, che si occupa di interpretare la parte di processo assegnatogli; l'esecuzione risulta distribuita e decentralizzata, infatti ogni robot esegue in modo autonomo il proprio processo rispettando il coordinamento descritto nella specifica globale [13].
Prendendo come riferimento la Tabella 3.1, l'approccio appena descritto viene riassunto nella Tabella 3.4.

Criterio	Descrizione
Nozione di coordinamento	Modellazione di un protocollo globale di collaborazione tramite un diagramma BPMN
Tipo di specifica adottata	Specifica grafica e dichiarativa basata su diagrammi di collaborazione BPMN
Meccanismo di realizzazione	Interpretazione del modello tramite motori di esecuzione BPMN locali
Distribuzione dell'esecuzione	Esecuzione distribuita e decentralizzata dei processi locali, partendo da una modellazione centralizzata del processo
Garanzie di correttezza	Assenza di sintesi corretta per costruzione del protocollo distribuito e di garanzie formali sulle proprietà della collaborazione

Tabella 3.4: Analisi dell'approccio FaMe.

3.5 Approcci di verifica formale delle collaborazioni

La modellazione delle collaborazioni tramite BPMN presenta il problema della coerenza tra la specifica globale delle interazioni e le implementazioni dei partecipanti, poiché bisogna verificare in modo formale che i diagrammi di collaborazione rispettino tutti i vincoli inizialmente imposti dalla coreografia [12]. Viene definita infatti una semantica operazionale per i choreography e collaboration diagrams, che si occupa di associare a ciascun modello un sistema di transizione etichettato, così da descriverne il comportamento dinamico; in questo modo al sistema è consentito di confrontare i comportamenti di ogni modello tramite relazioni di conformità [12].
La verifica di quest'ultima si può realizzare tramite relazioni basate su bisimulazione e su equivalenza di tracce, che sono in grado di rilevare comportamenti anomali dovuti a possibili deadlock e al non determinismo [12]. La conformità inoltre viene definita come relazione tra una coreografia e una collaborazione opportunatamente astratta, in modo da ignorare i comportamenti che non sono regolati dalla specifica globale [12].
Questo approccio ha come obiettivo quello di verificare se un modello collaborativo sia in grado di soddisfare le proprietà di correttezza rispetto alla proiezione sui singoli partecipanti, definendo criteri di conformità tra specifica globale e comportamento locale; si colloca dunque nell'ambito della verifica formale di modelli che sono stati già definiti, consentendo di stabilire se una collaborazione possa costituire una valida implementazione della coreografia iniziale.
In base ai criteri definiti nella Tabella 3.1, anche per questo approccio, è presente un'ulteriore analisi (Tabella 3.5).

Criterio	Descrizione
Nozione di coordinamento	Relazione formale di conformità tra coreografia globale e collaborazione
Tipo di specifica adottata	Modelli BPMN dotati di semantica operazionale formale
Meccanismo di realizzazione	Associazione dei modelli a sistemi di transizione etichettati e verifica tramite relazioni di conformità basate su bisimulazione ed equivalenza di tracce
Distribuzione dell'esecuzione	Architettura distribuita basata su processi che comunicano tramite messaggi
Garanzie di correttezza	La correttezza viene controllata tramite verifica formale su modelli già costruiti, senza meccanismi di sintesi automatica corretta per costruzione

Tabella 3.5: Analisi degli approcci di verifica formale.

3.6 Confronto e posizionamento del contributo

Le sezioni precedenti hanno messo in evidenza le differenze nell'ambito del coordinamento nei MRS, analizzando gli approcci basati sull'allocazione dei compiti, soluzioni Model-Driven e DSL, framework basati sul workflow e notazione BPMN, oltre a metodi di verifica formale della conformità tra la specifica globale e sistema distribuito.
Con lo scopo di mettere in evidenza tutte le analogie, differenze e limiti, gli approcci descritti precedentemente vengono confrontati nella Tabella 3.6, nella quale verrà riportato anche l'approccio proposto in questo lavoro di tesi, in modo da evidenziarne il proprio posizionamento rispetto alle altre soluzioni già presenti.

Approccio	Nozione di coordinamento	Tipo di specifica	Meccanismo di realizzazione	Distribuzione dell'esecuzione	Garanzie di correttezza
MRTA	Allocazione e pianificazione dei compiti	Rappresentazione basata su task e assegnazioni	Algoritmi decisionali	Distribuita o ibrida	Nessuna garanzia formale globale
Model-Driven/DSL	Modellazione e orchestrazione di missioni	Specifica dichiarativa multilivello	Trasformazioni da modello a modello e da modello a codice	Coordinamento spesso centralizzato con esecuzione distribuita	Assenza di verifica formale integrata
Workflow e BPMN (FaMe)	Modellazione di un protocollo globale di collaborazione	Diagrammi BPMN di collaborazione	Interpretazione tramite motori BPMN locali	Esecuzione distribuita a partire da modellazione globale centralizzata	Assenza di sintesi corretta per costruzione
Verifica formale BPMN	Relazione formale di conformità tra coreografia e collaborazione	BPMN con semantica operazionale formale	Associazione a LTS e verifica tramite bisimulazione e tracce	Distribuita (processi comunicanti)	Verifica ex-post su modelli già definiti
Approccio proposto in questo lavoro di tesi	Sintesi automatica di comportamenti collaborativi a partire da specifica globale	Specifica formale del protocollo collaborativo	Generazione automatica dei comportamenti locali	Distribuita per costruzione	Correttezza garantita per costruzione

Tabella 3.6: Confronto tra gli approcci.

Capitolo 4

Background teorico

In questo capitolo saranno approfonditi tutti i concetti teorici fondamentali che sono alla base dei comportamenti collaborativi per i MRS esplorati in questo lavoro di tesi.
Dopo l'introduzione iniziale ai MRS e alle relative architetture di coordinamento, in questa sezione verranno esplorate in modo dettagliato le coreografie, le loro specifiche, e le sfide legate alla loro realizzazione.
L'obiettivo è quello di stabilire un quadro teorico per permettere di comprendere il metodo automatico proposto per la collaborazione tra MRS (Capitolo 5).

4.1 Coreografie

4.1.1 Definizione di coreografia

Nel contesto affrontato in questo lavoro di tesi, possiamo dire che una coreografia è la descrizione globale delle interazioni tra i partecipanti di una missione. In particolare, ha il compito di: **stabilire** l'ordine dei messaggi da scambiare, **definire** i vincoli che devono essere rispettati e **specificare** le condizioni con cui i messaggi devono essere scambiati. L'idea quindi è quella di stabilire le regole con cui devono essere scambiati i messaggi, definendo in anticipo tutti i vincoli e tutte le relazioni che permettono lo scambio di informazioni tra i partecipanti [10].

4.1.2 Coreografie di servizi

La coreografia di servizi rappresenta le interazioni osservabili tra tutti i servizi che partecipano ad una missione. Quest'ultime sono descritte dal punto di vista di un osservatore che ha il compito di registrare i messaggi scambiati e la sequenza con cui avvengono tali scambi [16].
Definisce quindi un protocollo che stabilisce: il modo in cui devono cooperare i servizi, quali messaggi devono essere inviati (e quali no), e da chi devono essere inviati i messaggi in un preciso momento.
Le fasi illustrate in Figura 4.1 rappresentano il ciclo di vita di una coreografia di servizi,

ed includono: progettazione, distribuzione, monitoraggio e valutazione [15]. Queste rappresentano il modo in cui la coreografia viene definita, resa disponibile ai partecipanti alla missione e osservata durante l'esecuzione dell'intero sistema. Inoltre viene evidenziato anche il legame con le orchestrazioni dei singoli partecipanti alla missione, che devono implementare localmente il comportamento definito globalmente.

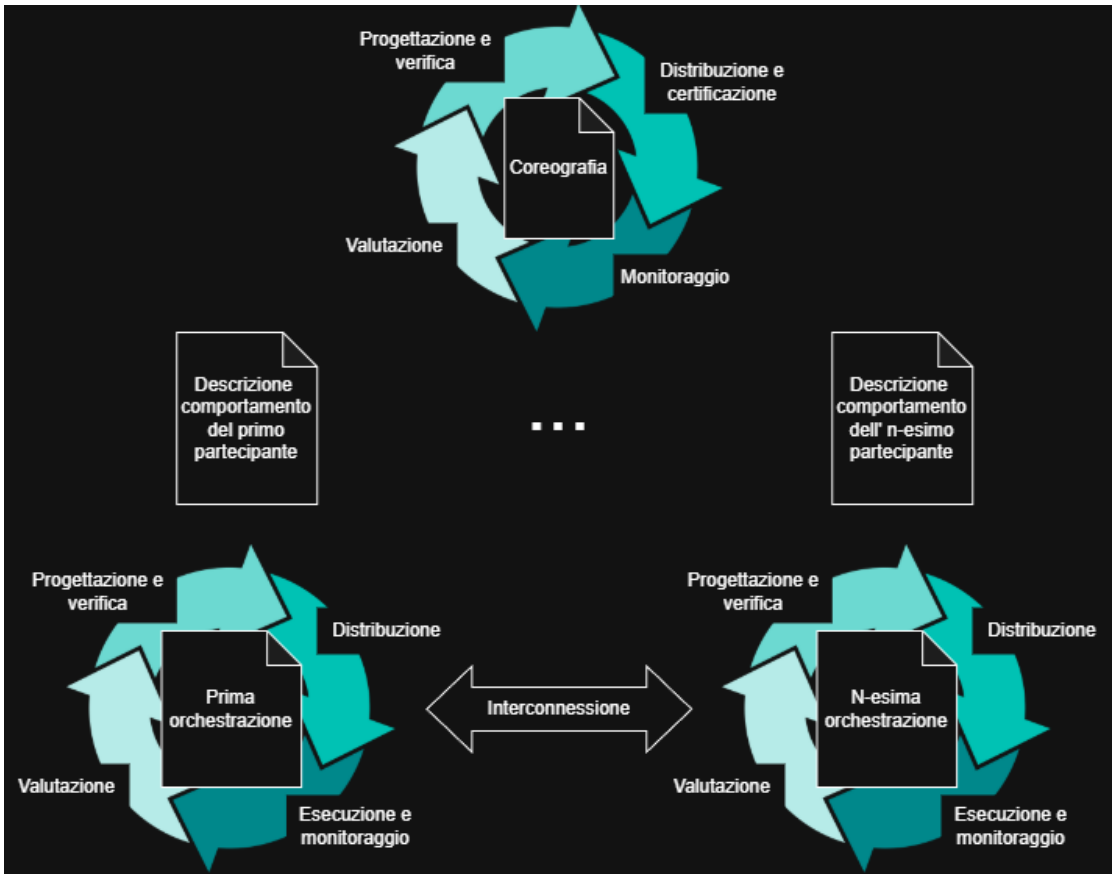


Figura 4.1: Ciclo di vita di una coreografia e relazione con le orchestrazioni dei partecipanti.

4.2 Specifica di coreografie

Dopo l'introduzione sul concetto generale di coreografia, coreografia di servizi e relativo ciclo di vita, questa sezione vedrà l'approfondimento delle modalità con cui una coreografia viene formalmente specificata. La specifica rappresenta il passaggio che permette di trasformare la descrizione informale delle interazioni in un modello vero e proprio.
Nelle seguenti sottosezioni verranno presentati tutti gli elementi che compongono una specifica di coreografia, partendo dalla definizione formale, passando per i costrutti di controllo, fino al processo di proiezione verso le orchestrazioni locali.

4.2.1 Coreografia come specifica formale

In una coreografia, il flusso di interazioni viene rappresentato con un modello che consente sia analisi che verifica del comportamento complessivo. Essa infatti rappresenta una

specifica formale del comportamento globale del sistema, ed è stata formalizzata come una macchina a stati, in cui questi ultimi sono considerati configurazioni del sistema. Le transizioni invece modellano tutti gli scambi di messaggi insieme alle strutture che permettono il controllo del flusso [3].
Il modo corretto per specificare una coreografia è quello specificato nella tupla 4.1 [3]:

$$\mathcal{C} = (\mathcal{P}, \mathcal{S}^c, s_0^c, \mathcal{L}, \mathcal{T}^c) \tag{4.1}$$

Simbolo	**Elemento della coreografia**
$\mathcal{P}$	Insieme finito dei partecipanti
$\mathcal{S}^c$	Insieme finito di stati
s_0^c	Stato iniziale
$\mathcal{L}$	Insieme finito delle etichette di messaggio
$\mathcal{T}^c$	Insieme delle transizioni

Tabella 4.1: Elementi della specifica di una coreografia.

La formalizzazione appena descritta rappresenta la base per l'analisi del comportamento globale, oltre ad essere considerata il punto di partenza per la derivazione di tutti i modelli di comportamento locali tramite il processo di proiezione, dal quale è possibile ottenere il comportamento atteso di ogni partecipante a partire dalla specifica globale [3].

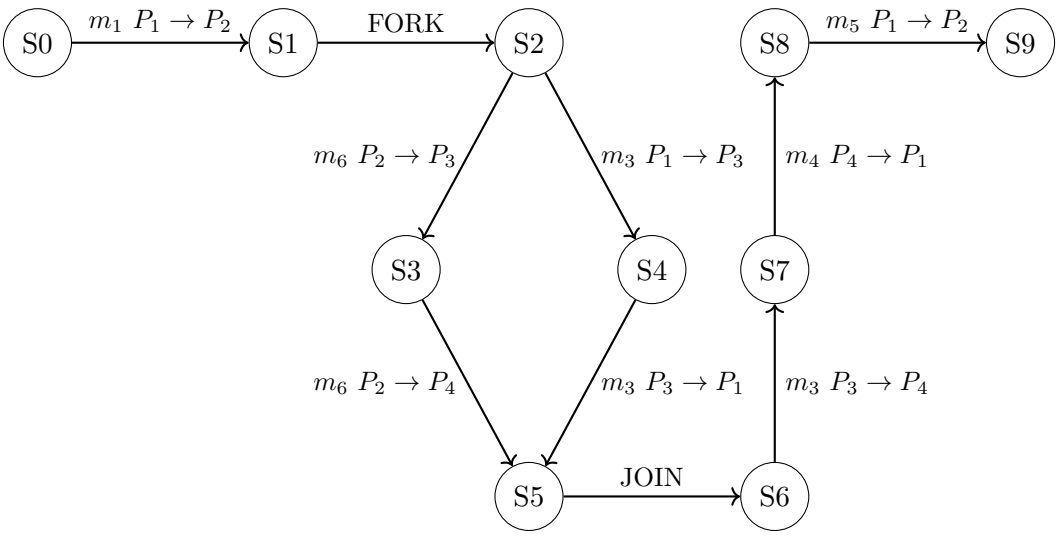


Figura 4.2: Esempio della specifica di una coreografia.

La Figura 4.2 rappresenta un semplice esempio di specifica di coreografia modellata, dove vengono rappresentati *sei* stati, *cinque* scambi di messaggi tra i partecipanti e i due costrutti *FORK* e *JOIN* che permettono il controllo del flusso. Per semplicità

notazionale, nella figura sono stati utilizzati simboli che differiscono dalla notazione introdotta in letteratura, ma che non ne alterano il significato.
Il comportamento della macchina a stati appena presentata è il seguente:

1. $S_0 \rightarrow S_1$: la coreografia inizia con lo scambio del messaggio *m1* tra il *primo* partecipante ed il *secondo*.
2. $S_1 \rightarrow S_2$: il flusso si divide in *due* rami paralleli tramite una transizione *fork*.
3. $S_2 \rightarrow S_3$ e $S_2 \rightarrow S_4$: vengono attivati due rami paralleli nel flusso, mediante gli scambi dei messaggi *m6* (tra il *secondo* partecipante ed il *terzo*), e *m3* (tra *primo* e *terzo* partecipante).
4. $S_3 \rightarrow S_5$ e $S_4 \rightarrow S_45$: i rami continuano il flusso con gli scambi dei messaggi *m6* (tra il *secondo* partecipante ed il *quarto*), e *m3* (tra *primo* e *terzo* partecipante). Dopo questi scambi, convergono nello stato S_5.
5. $S_5 \rightarrow S_6$: il flusso viene riunito tramite una transizione *join*.
6. $S_6 \rightarrow S_7$: il flusso sequenziale continua con lo scambio del messaggio *m3* tra il *terzo* partecipante ed il *quarto*.
7. $S_7 \rightarrow S_8$: scambio del messaggio *m4* tra il *quarto* partecipante ed il *primo*.
8. $S_8 \rightarrow S_9$: il comportamento globale termina con lo scambio del messaggio *m5* tra il *primo* ed il *secondo* partecipante.

4.2.2 Rappresentazione grafica della coreografia (BPMN 2.0)

Nella notazione BPMN 2.0 le coreografie sono modellate con diversi costrutti, ciascuno dei quali rappresenta le interazioni e i diversi scambi di messaggi tra i partecipanti, senza descriverne però i processi interni [23].
In questa sezione approfondiremo i concetti di:

- Choreography diagrams.
- Choreography task.
- Flussi di controllo utilizzati durante la modellazione (sequence flow, gateway, eventi intermediari, eventi di bordo).

4.2.2.1 Choreography diagrams e differenza con i collaboration diagrams

I choreography diagrams sono diagrammi che forniscono una rappresentazione globale di tutte le interazioni che avvengono tra i partecipanti all'interno di un sistema distribuito, esprimendo vincoli sulle interazioni e sugli scambi di messaggi tra i componenti del sistema; i collaboration diagrams invece, oltre alle diverse interazioni tra i partecipanti,

descrivono anche il comportamento interno di ogni singolo componente. Questo permette di fornire una rappresentazione più fedele possibile ad una futura implementazione dell'intero sistema [12].
Un esempio di questi diagrammi, relativi al caso di studio che viene presentato in questo lavoro di tesi, è presente nel Capitolo 5, rispettivamente nelle Sezioni 5.10 e 5.13.

4.2.2.2 Choreography task

Un choreography task rappresentà un'attività atomica all'interno di una coreografia BPMN che modella un'interazione basata sullo scambio di messaggi tra diversi partecipanti. Dal punto di vista grafico, il choreography task viene rappresentato con un rettangolo con i bordi arrotondati, suddiviso a sua volta in tre diversi rettangoli: nel rettangolo *superiore* c'è il partecipante che avvia l'interazione, in quello *inferiore* ci sono il/i destinatario/destinatari, ed infine in quello *intermedio* è presente il nome del task. Un choreography task può essere rappresentato da tre diversi marcatori interni [23]:

1. **Loop**: il task si ripete più volte, fino a quando una condizione del loop non diventa falsa.
2. **Tre linee verticali**: il task viene eseguito in parallelo con più istanze attive in contemporanea. Nel caso di studio presentato nel Capitolo 5 è presente un task di questo tipo.
3. **Tre linee orizzontali**: il task si ripete per ogni istanza prevista, una dopo l'altra.

4.2.2.3 Principali elementi della notazione BPMN 2.0

In questa sottosezione verranno elencati i principali elementi utilizzati in questo lavoro di tesi, secondo la specifica BPMN 2.0 [23]:

1. **End event**: indica il punto in cui il processo termina.
2. **Gateway**: utili per il controllo dei flussi di sequenza. Vengono inseriti per modellare possibili divergenze e convergenze.
3. **Intermediate event**: indica un evento che si verifica tra lo start event e l'end event.
4. **Message flow**: rappresenta lo scambio di messaggi tra due partecipanti distinti.
5. **Pool e Lane**: il primo riguarda la rappresentazione di un partecipante al processo; la seconda è una partizione del pool, utile ad organizzare le attività interne del processo.
6. **Start event**: indica il punto in cui il processo inizia.

7. **Sequence flow**: rappresenta l'ordine in cui le operazioni o gli elementi all'interno di un processo o coreografia vengono svolti.

8. **Service task**: è un'attività automatizzata eseguita dal sistema. Utilizzato per modellare operazioni svolte dai servizi.

9. **Timer intermediate event**: è un evento a tempo utilizzato per rappresentare un intervallo di tempo specificato prima di proseguire con il flusso di operazioni.

10. **Timer boundary event**: simile al timer precedente, ma è collegato ad un'attività anziché al flusso tramite sequence flow.

La Figura 4.3 associa ciascun elemento descritto alla sua rappresentazione grafica [23]:

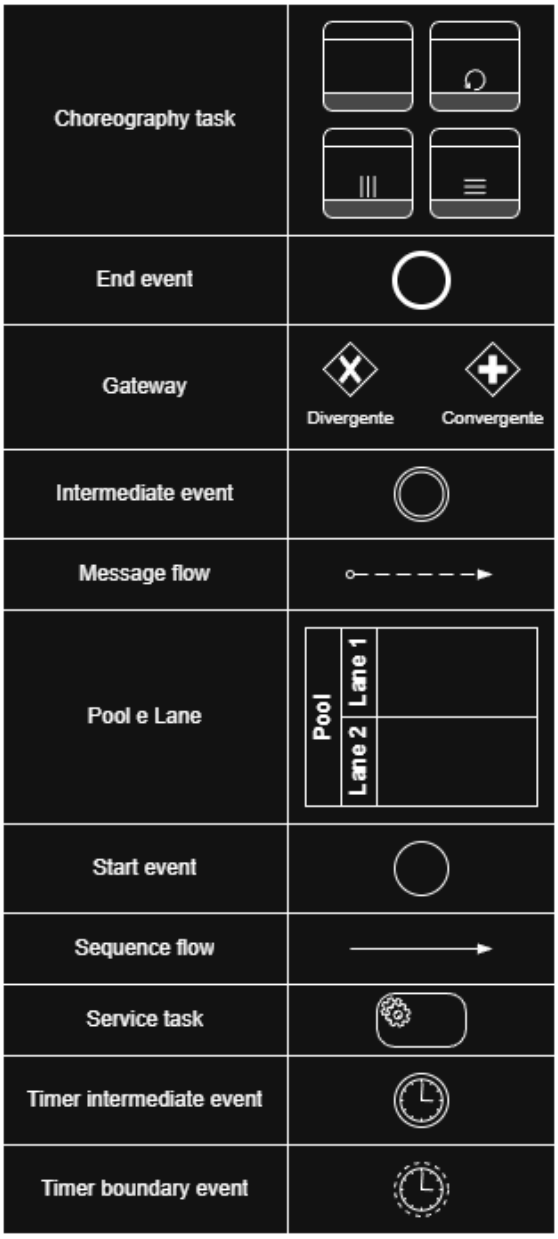


Figura 4.3: Principali elementi notazione BPMN 2.0.

4.2.3 Semantica della coreografia (stati e transizioni)

In questa sezione andremo a definire la semantica basata su tracce di una coreografia, descrivendo il comportamento globale del sistema in termini di sequenze di scambi di messaggi; si può dire infatti che una specifica di coreografia viene considerata una Labeled Transition System (LTS), estesa con differenti costrutti di controllo del flusso [3]. In particolare, una LTS viene rappresentata tramite la tupla 4.2 [9], spiegata nella Tabella 4.2:

$$\langle S, \Sigma, \rightarrow, \alpha_0, F \rangle \tag{4.2}$$

Simbolo	**Elemento della LTS**
S	Insieme degli stati
Σ	Alfabeto delle etichette
$\rightarrow$	Relazione di transizione etichettata
α_0	Stato iniziale
F	Insieme degli stati finali

Tabella 4.2: Elementi della LTS.

Quindi, facendo riferimento alla tupla 4.1, dato:

- Un insieme di partecipanti.
- Un insieme di stati.
- Uno stato iniziale.
- Un insieme di messaggi.
- Un insieme di transizioni.

L'alfabeto di una coreografia è definito come nell'insieme 4.3 [3]:

$$\Sigma_C = \{m_{P \rightarrow P'} \mid P, P' \in \mathcal{P} \wedge m \in \mathcal{L}\} \cup \{\mathit{fork}, \mathit{join}\}. \tag{4.3}$$

In una coreografia $\mathcal{C}$ una sequenza di transizioni è chiamata *path*, ed è formato da tutte le transizioni che sono state etichettate da scambi di messaggi, fork, oppure join. L'insieme di tutti i path presenti in una coreografia è chiamato $Path(c)$, ed è costituito da tutti micro-cammini, che hanno la forma descritta nella notazione di transizione 4.4 [3].

$$s^i_{\mathcal{C}} \xrightarrow{m_{P \rightarrow P'}} s^j_{\mathcal{C}} \;\mid\; s^i_{\mathcal{C}} \xrightarrow{\text{fork}} s^j_{\mathcal{C}} \;\mid\; s^i_{\mathcal{C}} \xrightarrow{\text{join}} s^j_{\mathcal{C}} \tag{4.4}$$

Come ultimo concetto abbiamo la *trace*, che rappresenta una sequenza di interazioni a partire dallo stato iniziale, e si denota con $L(C)$ [3].

A conclusione di questa parte, prendendo come riferimento la Figura 4.2, possiamo rappresentare in modo concreto quanto detto in queste sottosezioni:

- Gli **stati** della coreografia sono indicati come: $S_0, S_1, S_2, S_3, S_4, S_5, S_6, S_7, S_8$ e S_9.
- Un **path** possibile che parte da S_0 è indicato nella notazione 4.5:

$$S_0 \xrightarrow{m_1\, P_1 \to P_2} S_1 \xrightarrow{\text{fork}} S_2 \xrightarrow{m_3\, P_1 \to P_3} S_4 \xrightarrow{m_3\, P_3 \to P_1} S_5 \xrightarrow{\text{join}} S_6 \tag{4.5}$$

- Al path descritto al punto precedente corrisponde una **sequenza di interazioni**, rappresentata nell'espressione 4.6:

$$m_1\, P_1 \to P_2,\ \text{fork},\ m_3\, P_1 \to P_3,\ m_3\, P_3 \to P_1,\ \text{join} \tag{4.6}$$

- Una possibile **trace** che appartiene all'insieme $L(C)$ invece è rappresentata dalla sequenza di interazioni presente nell'espressione 4.7:

$$m_1\, P_1 \to P_2,\ \text{fork},\ m_6\, P_2 \to P_3,\ m_6\, P_2 \to P_4,\ \text{join},\ m_3\, P_3 \to P_4 \tag{4.7}$$

4.2.4 Tipi di stati nella coreografia

In questa sezione vedremo le quattro differenti tipologie di stati [3], ciascuna dei quali ha caratteristiche differenti in termini di transizioni in entrata e in uscita. In particolare gli stati si suddividono in: **plain**, **branch**, **fork** e **join**. Prima di iniziare con il primo tipo, nella Tabella 4.3 viene definita la notazione che verrà adottata durante la spiegazione di ciascuno di esso [3].

Simbolo	Significato
$\lvert S_i^C \rvert_{\text{ex}}^{\text{in}}$	Numero di transizioni di scambio in entrata nello stato S_i^C
$\lvert S_i^C \rvert_{\text{ex}}^{\text{out}}$	Numero di transizioni di scambio in uscita dallo stato S_i^C
$\lvert S_i^C \rvert_{f_{\text{ork}}}^{\text{in}}$, $\lvert S_i^C \rvert_{f_{\text{ork}}}^{\text{out}}$	Numero di transizioni di tipo fork in entrata e in uscita dallo stato S_i^C
$\lvert S_i^C \rvert_{join}^{\text{in}}$, $\lvert S_i^C \rvert_{join}^{\text{out}}$	Numero di transizioni di tipo join in entrata e in uscita dallo stato S_i^C

Tabella 4.3: Cardinalità delle transizioni.

La rappresentazione del **plain state** è riportata e spiegata nella Tabella 4.4 [3]:

Notazioni	Significato
$\lvert S_i^C \rvert_{\text{ex}}^{\text{out}} \leq 1$	Lo stato S_i^C può avere al massimo una transizione di scambio in uscita
$\lvert S_i^C \rvert_{f_{\text{ork}}}^{\text{in}} = \lvert S_i^C \rvert_{j_{\text{oin}}}^{\text{out}} = 0$	Non ci possono essere né transizion fork entranti né transizioni join uscenti
$\lvert S_i^C \rvert_{f_{\text{ork}}}^{\text{out}} \leq 1$	Massimo una transizione fork uscente
$\lvert S_i^C \rvert_{j_{\text{oin}}}^{\text{in}} \leq 1$	Massimo una transizione join entrante

Tabella 4.4: Plain state.

Il **branch state** è riportato nella Tabella 4.5, dove è possibile notare che l'unica cosa che lo differenzia dallo stato precedente, è la prima notazione [3]:

Notazioni	Significato
$\lvert S_i^C \rvert_{\text{ex}}^{\text{out}} \geq 2$	Lo stato S_i^C deve avere almeno due transizioni di scambio in uscita
$\lvert S_i^C \rvert_{f_{\text{ork}}}^{\text{in}} = \lvert S_i^C \rvert_{j_{\text{oin}}}^{\text{out}} = 0$	Non ci possono essere né transizioni fork entranti né transizioni join uscenti
$\lvert S_i^C \rvert_{f_{\text{ork}}}^{\text{out}} \leq 1$	Massimo una transizione fork uscente
$\lvert S_i^C \rvert_{j_{\text{oin}}}^{\text{in}} \leq 1$	Massimo una transizione join entrante

Tabella 4.5: Branch state.

La definizione del **fork state** è presentata nella Tabella 4.6 [3]:

Notazioni	Significato
$\lvert S_i^C \rvert_{\text{ex}}^{\text{in}} = \lvert S_i^C \rvert_{\text{join}}^{\text{in}} = 0$	Lo stato S_i^C non può avere transizioni di scambio e di join in entrata
$\lvert S_i^C \rvert_{f_{\text{ork}}}^{\text{in}} = 1$	Ci deve essere esattamente una transizione fork in entrata
$\lvert S_i^C \rvert_{\text{ex}}^{\text{out}} + \lvert S_i^C \rvert_{f_{\text{ork}}}^{\text{out}} \geq 2$	Deve avere almeno due flussi in uscita
$\lvert S_i^C \rvert_{\text{join}}^{\text{out}} = 0$	Non può avere nessuna transizione join in uscita

Tabella 4.6: Fork state.

L'ultimo stato, il **join**, è descritto nella Tabella 4.7 [3]:

Notazioni	Significato
$\lvert S_i^C \rvert_{\text{ex}}^{\text{out}} = \lvert S_i^C \rvert_{f_{\text{ork}}}^{\text{out}} = 0$	Lo stato S_i^C non può avere transizioni di scambio e transizioni fork in uscita
$\lvert S_i^C \rvert_{\text{join}}^{\text{out}} = 1$	Deve avere esattamente una transizione join in uscita
$\lvert S_i^C \rvert_{\text{ex}}^{\text{in}} + \lvert S_i^C \rvert_{\text{join}}^{\text{in}} \geq 2$	Deve avere almeno due flussi in entrata
$\lvert S_i^C \rvert_{f_{\text{ork}}}^{\text{in}} = 0$	Non può avere una transizione fork in entrata

Tabella 4.7: Join state.

Facendo riferimento alla Figura 4.2, è possibile classificare gli stati della coreografia:

- $S_0, S_1, S_3, S_4, S_6, S_7, S_8$ e S_9 sono *plain states*.
- S_2 è un *fork state*.
- S_5 è un *join state*.
- Non sono presenti *branch state*, poiché nessuno dei dieci stati rispetta le condizioni richieste.

4.3 Ulteriori notazioni di specifica utili nel contesto dei MRS

4.3.1 Modelli di comportamento e specifica dei compiti nei MRS

In un sistema come quello descritto in questo lavoro di tesi, il comportamento globale è dato dall'interazione dei comportamenti dei singoli robot. Ognuno di questi infatti può essere modellato come un'entità autonoma che esegue il proprio flusso di attività operative e comunica con gli altri componenti utilizzando gli scambi di messaggi [13].
La missione complessiva del sistema può essere considerata come un insieme di compiti, anche detti task, ciascuno dei quali costituisce un sotto-obiettivo necessario al raggiungimento dell'obiettivo globale. Il processo di assegnazione di tali compiti ai robot è formalizzato come **Multi-Robot Task Allocation (MRTA)**; in questo ambito, i task possono essere distinti in [20]:

1. **Discreti**, ossia riferiti ad attività che terminano una volta raggiunto un determinato obiettivo.
2. **Continui**, relativi ad attività che si protraggono nel tempo .

Il comportamento del singolo robot viene considerato come un processo che contiene diverse *azioni interne* (che riguardano l'esecuzione dei task), ma anche *azioni di comunicazione* (vale a dire l'invio e la ricezione di informazioni).

Questo tipo di distinzione tra le due tipologie di azioni è coerente con l'astrazione adottata nel processo di proiezione di una coreografia, dove il comportamento locale proveniente dalla specifica globale costituisce l'insieme di tutte le azioni interattive che il partecipante deve rispettare [3].

4.3.2 Modelli di interazione e coordinamento tra robot

In questi sistemi il coordinamento nasce dalla necessità di ogni componente di tener conto delle azioni degli altri robot, in modo tale da ottenere un comportamento complessivo coerente. Ogni robot, a seconda del livello di conoscenza che possiede sugli altri, può essere considerato come **aware** o **unaware** [19]: il primo se possiede informazioni sui loro compagni di squadra, mentre il secondo se svolge i suoi compiti senza sapere cosa fanno gli altri robot all'interno del sistema.
Il coordinamento tra robot, invece, può essere classificato come [19]:

- **Forte**, se si basa su un protocollo esplicito che guida tutte le azioni dei robot
- **Debole**, quando il protocollo da seguire non è specificato e quindi la collaborazione si evidenzia tramite il comportamento individuale.

Questi concetti costituiscono la base per descrivere i modelli di interazione e coordinamento nei MRS, permettendo di comprendere come il comportamento individuale di ciascun robot contribuisca alla realizzazione della collaborazione.

4.3.3 Relazione tra specifica di missione e coreografie

Nei MRS, come descritto anche nelle sezioni precedenti, le missioni specificano la necessità di collaborazione tra più componenti per portare a termine dei compiti che un singolo robot non sarebbe in grado di svolgere; a tal proposito, con l'impiego di molteplici robot, le prestazioni dell'intero sistema aumenterebbero [19].
La missione quindi definisce l'obiettivo e l'insieme delle attività da realizzare; al contrario però non definisce e non specifica le modalità di interazione, che permettono la comunicazione tra robot.
La definizione di tali interazioni, come introdotto nella Sezione 4.1.1, avviene tramite la coreografia, che descrive tutti gli scambi di messaggi che avvengono tra i partecipanti, grazie a vincoli e condizioni ben specifiche.

4.4 Realizability

Questa sezione introduce il concetto di realizzabilità di una specifica globale, partendo da un'iniziale definizione formale, per poi analizzare le principali cause di non realizzabilità, ed infine discuterne l'applicazione nel contesto di questo lavoro di tesi, i multi-robot systems.

Il problema più importante di una specifica globale è quello di determinare se essa può essere realizzata, ovvero se esistono più entità distribuite le cui interazioni siano conformi alla specifica stessa [5].
La realizzabilità di una coreografia non è una proprietà automatica; infatti una sua descrizione può risultare corretta sintatticamente, al contrario però può non essere implementabile in un sistema distribuito dove i partecipanti comunicano in modo asincrono. La verifica della realizzabilità stabilisce se la specifica globale ammette una realizzazione distribuita, equivalente al comportamento descritto [5].

4.4.1 Cause di non realizzabilità

Una specifica globale è condiderata non realizzabile se il comportamento all'interno del sistema distribuito, precedentemente ottenuto con la proiezione sui partecipanti, non è lo stesso di quello descritto dalla coreografia. La realizzabilità quindi richiede equivalenza linguistica tra la specifica e il sistema asincrono derivato dalle proiezioni; questa condizione è descritta nell'espressione 4.8 e spiegata nella Tabella 4.8 [5]:

$$L(C) = L(\text{DETER}(I_1^c)) \tag{4.8}$$

Simbolo	Significato
C	Specifica globale della coreografia
$L(C)$	Insieme delle sequenze di messaggi ammesse dalla specifica globale
I_1^c	Sistema asincrono ottenuto tramite la proiezione della coreografia sui partecipanti, dove ogni canale di comunicazione contiene massimo un messaggio
$DETER$	Indica la determinizzazione del sistema proiettato (sistema in cui da uno stato, una determinata azione conduce al più ad uno stato successivo)
$L(\text{DETER}(I_1^c))$	Linguaggio generato dal sistema distribuito

Tabella 4.8: Equivalenza linguistica.

Se questa uguaglianza non è rispettata ($L(C) \neq L(\text{DETER}(I_1^c))$), allora tale violazione costituisce una causa di non realizzabilità, poiché implicherebbe che il sistema generi sequenze di messaggi che non erano previste per la specifica globale, oppure che esso non riproduca delle sequenze che erano state precedentemente accettate dalla specifica.
Una possibile seconda causa di non realizzabilità invece è legata alla comunicazione asincrona: nel modello considerato, infatti, i partecipanti comunicano tramite l'invio e

la ricezione di messaggi su canali FIFO (First-In First-Out, quindi nell'ordine di arrivo) con capacità potenzialmente illimitate [5]. Dal momento in cui la configurazione del sistema globale include il contenuto delle code appena descritte, può accadere che si accumuli un numero arbitrario di messaggi; ciò implica che il numero degli stati possa diventare infinito, nonostante i partecipanti abbiano il comportamento modellato come macchina a stati finiti [5].
Infine, la realizzabilità richiede che il sistema proiettato abbia la condizione di ben-formazione (well-formedness) soddisfatta. Questa proprietà è formalizzata tramite la formula CTL 4.9; il significato dei simboli utilizzati invece è riportato nella Tabella 4.9 [5].

$$AG\big(|Q_i| > 0 \Rightarrow EF(|Q_i| = 0)\big) \tag{4.9}$$

Simbolo	Significato
Q_i	Coda di comunicazione associata al partecipante i
$\|Q_i\|$	Numero di messaggi presenti nella coda del partecipante i
AG	Indica che la proprietà deve valere in tutti gli stati raggiungibili
EF	Indica che esiste un'evoluzione futura in cui la condizione è soddisfatta

Tabella 4.9: Well formedness.

La Formula 4.9 sta a significare che in ogni stato raggiungibile del sistema, se la coda è riempita, esiste almeno un momento in cui la coda dei messaggi in futuro si svuoti [5]. Se questa proprietà venisse violata, allora esisterebbe uno stato in cui un messaggio dentro la coda non può essere consumato. La specifica globale quindi risulterebbe non realizzabile, poiché il sistema non soddisferebbe la condizione di well-formedness [5].

4.4.2 Realizability nei MRS

La nozioni introdotte nelle Sezioni 4.4 e 4.4.1 possono essere applicate nel contesto dei MRS; in particolare, la missione globale viene considerata una coreografia, mentre i robot sono partecipanti che eseguono comportamenti locali. La specifica globale della missione descrive quindi l'intera sequenza di interazioni, incluse le comunicazioni tra i robot; il sistema distribuito è invece costituito da una flotta di robot autonomi, ciascuno dei quali si coordina tramite lo scambio di messaggi senza l'ausilio di un'entità centralizzata.
Per essere realizzabile, il comportamento generato dall'esecuzione deve essere lo stesso di quello descritto dalla missione globale, seguendo la nozione di equivalenza linguistica precedentemente introdotta. Nei sistemi multi-robot, se le interazioni tra componenti producono sequenze di azioni non previste, oppure se alcune sequenze non vengono ese-

guite a causa di vincoli o asincronia, tale corrispondenza può essere compromessa.
La condizione di well-formedness invece assume un significato prettamente operativo: gli eventi ed i messaggi generati dai robot devono poter essere gestiti in futuro dal destinatario. Se ciò non dovesse accadere, ovvero se un componente resta in attesa per un tempo indefinito, allora la realizzabilità verrà compromessa.
Quindi, una coreografia può essere realizzabile dal punto di vista formale, al contrario però può non esserlo nel mondo fisico. Possibili ritardi, incertezze dei robot e fallimenti fisici possono compromettere sincronizzazione, scelte corrette di rami e sequenze di interazioni. Per questi motivi, applicare le coreografie ai MRS richiede meccanismi di coordinamento in grado di compensare ritardi, incertezze e fallimenti, altrimenti costituirebbero cause di non realizzabilità.

4.5 Problemi nella realizzazione delle coreografie

4.5.1 Comunicazione sincrona e asincrona

La realizzazione di una coreografia dipende dalla semantica di comunicazione adottata, infatti le differenze tra comunicazione sincrona ed asincrona possono incidere sulle modalità con cui vengono scambiati i messaggi; in particolare, la presenza o meno delle code, e delle condizioni di blocco, cambiano la gestione dei messaggi durante l'esecuzione distribuita.
Nella Tabella 4.10 è riportato un confronto tra le due modalità, in base a: *modalità di interazione*, *invio e ricezione del messaggio*, *condizioni di blocco send e receive*, *relazione tra send e receive* e *spazio degli stati* [3, 5].

Aspetto	Com. sincrona	Com. asincrona
Modalità di interazione	Invio e ricezione accoppiati, consumo immediato del messaggio	Invio con accodamento in FIFO, ricezione tramite consumo dalla testa
Invio del messaggio	Il messaggio viene consumato immediatamente dal destinatario	Il messaggio viene inserito nella coda del destinatario
Ricezione del messaggio	Contemporanea all'invio	Il messaggio viene consumato dalla testa della coda
Relazione tra send e receive	Azioni modellate congiuntamente	Azioni modellate separatamente
Condizioni di blocco (send)	—	Invio bloccante se la coda del destinatario è piena
Condizioni di blocco (receive)	—	Ricezione bloccante se la coda è vuota o se in testa è presente un messaggio diverso da quello atteso
Spazio stati	Finito (assenza di code)	Potenzialmente infinito

Tabella 4.10: Confronto tra comunicazione sincrona e asincrona.

4.5.2 Deadlock e situazioni di stallo

In una coreografia, durante l'esecuzione, possono verificarsi situazioni di stallo, ovvero delle configurazioni globali dove nessuna azione è abilitata, anche se nessuno stato finale previsto dalla specifica è stato raggiunto; in questi casi i partecipanti smettono di interagire tra di loro, quindi il sistema non è in grado di proseguire l'esecuzione inizialmente prevista.
Nel modello di comunicazione asincrona, queste situazioni appena descritte possono essere causate dalle azioni di invio e ricezione. In particolare [3]:

- L'operazione di invio di messaggi può risultare bloccata quando la coda del destinatario è piena.
- La ricezione dei messaggi può fermarsi se la coda è piena o se nella testa della coda c'è un messaggio differente da quello atteso.

Nei sistemi concorrenti, l'esecuzione deriva dall'ordine in cui le azioni vengono svolte in parallelo dai partecipanti (*interleaving*); senza meccanismi di coordinamento adeguati, tali interleaving possono condurre a sistemi bloccati (*deadlock*) [11].
La Figura 4.4 mostra un esempio di una configurazione che porta ad un deadlock.

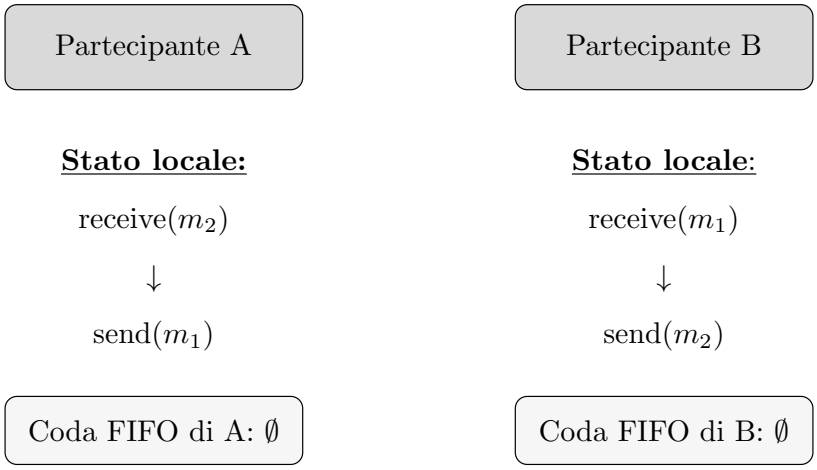


Figura 4.4: Esempio di una situazione di stallo causata da attesa reciproca.

Nella configurazione mostrata in figura, i due partecipanti hanno entrambi come prima operazione la ricezione; dal momento in cui entrambe le code risultano vuote, nessuna delle due è abilitata e le azioni di invio non si trovano nella testa della coda. È come se il *partecipante A* si chiedesse *"posso fare il receive di m2?"*. La risposta è negativa poiché nella coda di A non è presente *m2*; stesso discorso per il *partecipante B* con il *messaggio m1*. Il sistema quindi si trova in uno stato senza alcuna transizione possibile. Queste situazioni evidenziano come come le situazioni di stallo non sono riconducibili soltanto a degli errori sintattici nella specifica globale, ma possono essere evidenziate anche grazie alla dinamica delle interazioni tra i partecipanti.
Dal punto di vista coreografico, una situazione di stallo si verifica quando non vengono generate correttamente le sequenze di messaggi inizialmente previste dalla specifica; il comportamento osservato non è lo stesso di quello descritto a livello globale, dal momento in cui il sistema si arresta in una configurazione intermedia [5].
Alcuni approcci affrontano questo problema garantendo la libertà da situazioni di stallo (*deadlock-freedom*) già durante la fase di progettazione della coreografia, così che l'esecuzione delle azioni non conduca verso stati bloccanti [11].

4.5.3 Inconsistenze di ordine e perdita di messaggi

Nei sistemi asincroni, durante il passaggio tra la specifica globale e il comportamento eseguito dai partecipanti, possono verificarsi due principali problematiche:

1. **Inconsistenze di ordine**: si verificano quando il sistema genera sequenze aventi tacce che non rispettano l'ordine previsto dalla specifica globale [3]. Anche se ogni partecipante opera correttamente a livello locale, la sequenza di messaggi osservata può comunque essere differente da quella stabilita dalla coreografia.

2. **Perdita dei messaggi**: in questo tipo di sistema ogni singolo messaggio inviato deve essere ricevuto in almeno un percorso di esecuzione [5], e se ciò non accade,

esso rimane nella coda impedendo al sistema di proseguire in uno stato ammesso dalla specifica. Quindi, nonostante il messaggio sia stato correttamente inviato, l'interazione prevista durante la configurazione non può essere completata e il sistema non realizza la sequenza di interazioni prevista dalla coreografia [3].

I sistemi con comunicazione asincrona possono introdurre quindi contrasti tra la semantica ed il comportamento effettivo del sistema; per rispettare l'ordine previsto dalla coreografia sono necessarie diverse tecniche di controllo.

4.6 Strategie per affrontare la realizzabilità delle coreografie

4.6.1 Verifica/analisi statica prima dell'esecuzione

Una strategia utile nel contesto di realizzabilità di una coreografia consiste in una sua analisi statica durante la progettazione, che si basa sul confronto tra il suo comportamento globale ed i comportamenti ottenuti durante la fase di proiezione [5]. In questo modo è possibile verificare se esiste effettivamente un insieme di partecipanti che riesce a generare tutte le sequenze di messaggi previste dalla specifica globale, oppure se tramite questa analisi emergono una o più sequenze aggiuntive, rendendo così necessaria una modifica prima dell'effettiva implementazione. I criteri utilizzati per questa verifica sono l'*equivalenza linguistica* tra la coreografia ed il sistema ottenuto, e la condizione di *well-formed* [5] (già discusse nella Sezione 4.4.1).
Un limite importante dell'analisi statica è che la sua complessità cresce all'aumentare del numero dei partecipanti, portando così al rifiuto della coreografia e alla richiesta di una sua riparazione, concetto che verrà affrontato nella Sezione 4.6.2.

4.6.2 Repair della coreografia

Nel momento in cui una coreografia risulta non realizzabile, è possibile modificare automaticamente la specifica globale in modo tale da ottenere una versione che riesca a soddisfare le condizioni di realizzabilità, con lo scopo di trasformare una specifica globale non realizzabile in una nuova che può essere implementata correttamente. Questo approccio può essere applicato in modo iterativo fino all'ottenimento di una coreografia corretta, e prende il nome di *repair della coreografia* [4], confrontato con la verifica/analisi statica [5] nella Tabella 4.11.

Aspetto	Verifica/analisi statica	Repair della coreografia
Obiettivo	Stabilire se la coreografia è realizzabile	Trasformare una coreografia non realizzabile in una realizzabile
Intervento sulla specifica	Nessuna modifica	Modifica automatica della coreografia
Momento di applicazione	Prima dell'esecuzione	
Risultato in caso negativo	Rifiuto della coreografia	Produzione di una versione riparata
Costo a run-time	Nessun overhead	

Tabella 4.11: Confronto tra verifica/analisi statica e repair della coreografia.

Il repair della coreografia è suddiviso in due principali strategie [4]:

1. **Relaxation**, dove l'insieme delle sequenze ammesse nella coreografia viene ampliato; in questo modo vengono inclusi anche i possibili comportamenti e le sequenze di interazioni che il sistema può generare in futuro. La nuova coreografia C' quindi avrà incluso al suo interno il linguaggio della coreografia originale ($L(C) \subseteq L(C')$).

2. **Restriction**, che ha un approccio opposto a quello della precedente strategia. Non amplia l'insieme delle sequenze ammesse, ma interviene sul comportamento del sistema aggiungendo vincoli o messaggi supplementari, con lo scopo di impedire la generazione di sequenze non previste e di mantenere così il linguaggio originale.

La Tabella 4.12 mostra un confronto tra le due strategie [4].

Aspetto	Relaxation	Restriction
Idea di base	Integra nella specifica globale comportamenti aggiuntivi emersi dal sistema proiettato	Impedisce comportamenti non previsti introducendo vincoli o messaggi di sincronizzazione
Effetto sul linguaggio	Amplia il linguaggio	Preserva il linguaggio originario (non accetta nuove sequenze della specifica)
Tipo di intervento	Modifica la coreografia accettando nuove sequenze	Aggiunge sincronizzazioni e/o vincoli alla coreografia per evitare sequenze indesiderate
Filosofia	Accettare sequenze in più	Eliminare sequenze non ammesse

Tabella 4.12: Confronto tra relaxation e restriction.

Le due strategie appena descritte sono formalizzate tramite un unico algoritmo, chiamato $Repair(C, inputRepairMechanism)$ [4], che viene riportato di seguito con il relativo pseudocodice, in cui viene evidenziato appositamente il ruolo del parametro *inputRepairMechanism*, che consente quindi di selezionare la strategia di repair.

Algoritmo Repair(C, inputRepairMechanism)

1: Calcola I_1^C
2: **if** $L(C) = L(I_1^C)$ **then**
3: **return** C
4: **end if**
5: Trova una differenza tra C e I_1^C
6: **if** inputRepairMechanism = *relaxation* **then**
7: Estendi C includendo la sequenza individuata
8: **else if** inputRepairMechanism = *restriction* **then**
9: Introduci vincoli o messaggi di sincronizzazione in C
10: **end if**
11: Ottieni la nuova coreografia C'
12: **GOTO** Riga 1 assegnando C a C'

1. **Riga 1**: viene costruito il sistema distribuito ottenuto tramite proiezione di C.
2. **Righe 2-4**: se il linguaggio della coreografia iniziale è lo stesso di quello del sistema proiettato, la coreografia risulta realizzabile e l'algoritmo termina.
3. **Riga 5**: se i due linguaggi invece non coincidono, allora esiste almeno una sequenza che appartiene al linguaggio del sistema proiettato ma non a quello della coreografia.
4. **Righe 6-10**: se il parametro passato a *inputRepairMechanism* è *relaxation* il linguaggio iniziale della coreografia viene ampliato; se invece è impostato su *restriction* allora vengono introdotti vincoli nuovi che impediscono la creazione di sequenze non ammesse.
5. **Riga 11**: la nuova coreografia C' sostituisce quella iniziale.
6. **Riga 12**: si torna alla riga 1 continuando finché non esistono più differenze tra i due linguaggi delle coreografie.

Questa parte si conclude con un esempio di una coreografia non realizzabile che fa riferimento alla Figura 4.2 della Sezione 4.2.1, leggermente rivisitata. Supponiamo di avere inizialmente un vincolo globale di precedenza (freccia tratteggiata rossa) tra le interazioni appartenenti ai due rami paralleli del fork, imponendo al sistema che $m3$

deve precedere ($\prec$) $m6$, come mostrato in Figura 4.5a; questo vincolo non è garantito dal modello asincrono di comunicazione, e può causare la non realizzabilità.

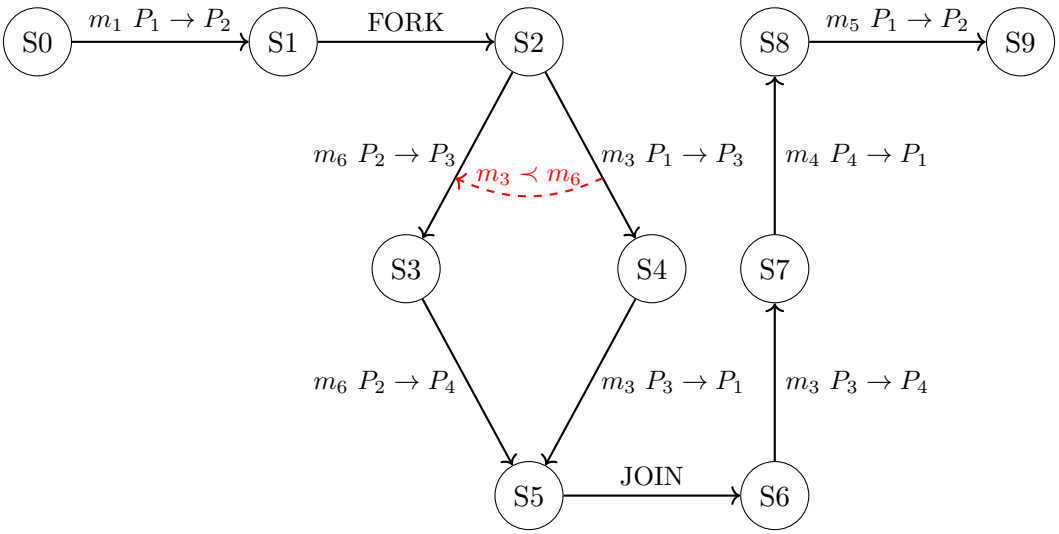


Figura 4.5a: Esempio della specifica di una coreografia.

I due rami generati dal *fork* possono fare interleaving in modo arbitrario, infatti non esiste nessun meccanismo che consente di verificare che l'interazione $m3$ venga eseguita prima di $m6$. Il sistema può generare una sequenza dove $m6$ precede $m3$, e quindi $L(C) \neq L(I_1^C) \implies$ coreografia non realizzabile.
Applicando l'algoritmo sopra citato, è possibile ripristinare la realizzabilità. Le Figure 4.5b e 4.5c mostrano rispettivamente la medesima coreografia con l'applicazione delle strategie di *relaxation* e *restriction*.

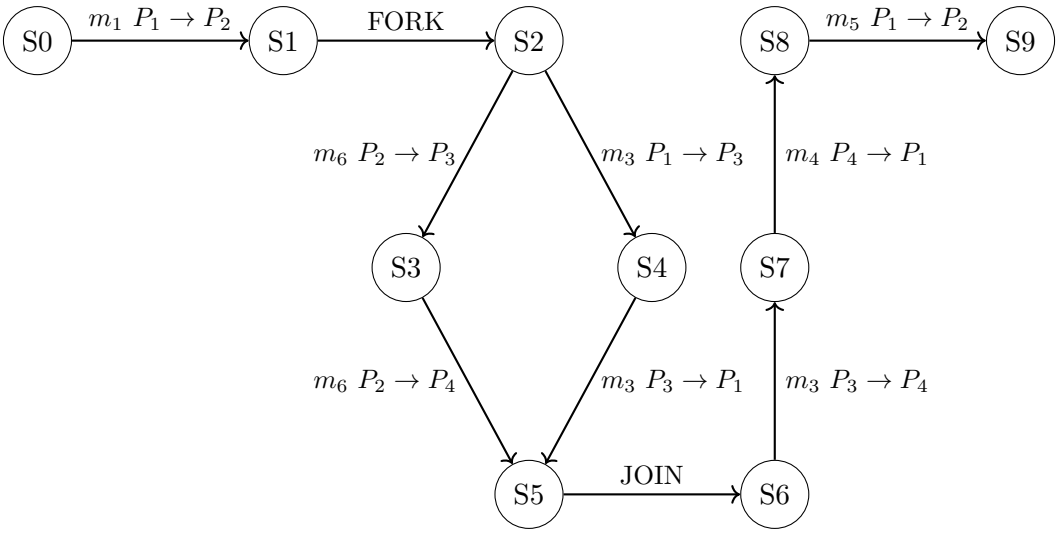


Figura 4.5b: Repair tramite relaxation (rimozione vincolo precedenza).

Nel caso della strategia di *rilassamento* il vincolo viene rimosso, con la coreografia risultante che, dopo la rimozione del vincolo di precedenza, coinciderebbe con quella originale mostrata in Figura 4.2. Questo permette di ampliare l'insieme delle sequenze

che la specifica ammette; in questo modo la coreografia accetta sia che $m3$ venga eseguito prima di $m6$, che viceversa, e si ottiene $L(C) \subseteq L(C')$, ripristinando così la realizzabilità.

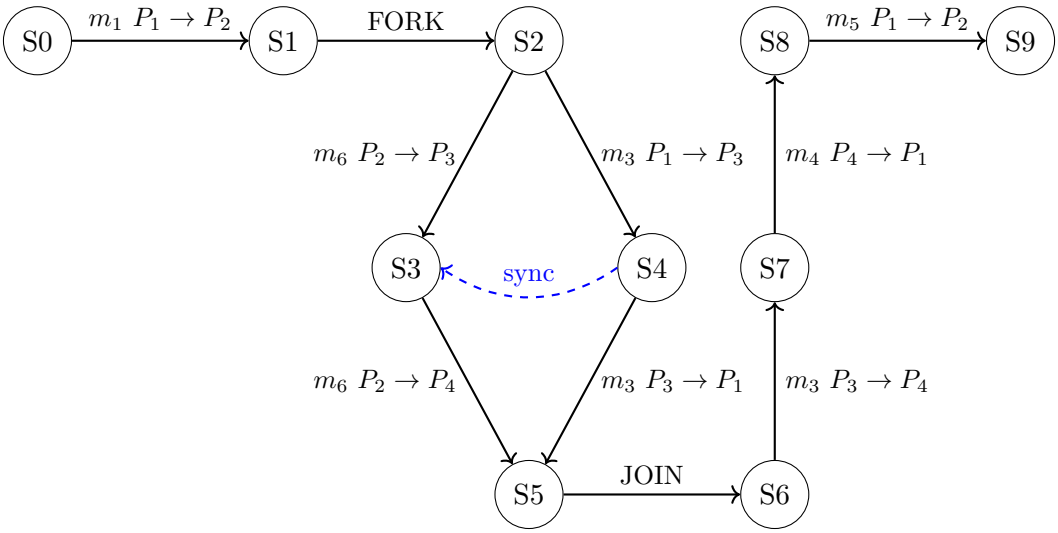


Figura 4.5c: Repair tramite restriction (introduzione sincronizzazione).

Nel caso invece della strategia di *restrizione* viene introdotto un meccanismo di coordinamento tra i partecipanti (sync) che garantisce l'esecuzione di $m6$ subito dopo quella di $m3$, impedendo così la generazione di sequenze non ammesse e ripristinando la realizzabilità della coreografia.

4.6.3 Enforcement a run-time

Oltre alle strategie discusse nelle Sezioni 4.6.1 e 4.6.2, un'ulteriore tecnica adottata nel contesto della realizzabilità consiste nell'introduzione dell'*enforcement a run-time*, un meccanismo che agisce durante l'esecuzione del sistema con lo scopo di garantire il rispetto della specifica globale della coreografia da parte delle interazioni tra i componenti della missione [1].
In generale, questo meccanismo è ottenuto tramite l'introduzione di componenti di controllo o di mediazione (che saranno approfonditi in 4.7.1), che si interpongono tra i partecipanti regolando così il flusso delle loro comunicazioni [1, 2].
L'enforcement a run-time consente pertanto di garantire la conformità con la specifica globale senza intervenire in modo diretto sul comportamento locale del singolo partecipante.

4.7 Coordination Delegates

In questa parte verrà approfondito il concetto di come i Coordination Delegates, definiti inizialmente nella Sezione 2.4 e richiamati indirettamente in 4.6.3, intervengono nelle interazioni dei vari partecipanti; verrà inoltre esplicato il ruolo che hanno durante

l'enforcement della coreografia, ed il loro utilizzo quando i partecipanti della missione vengono considerati componenti black-box.

4.7.1 Ruolo dei CDs nell'enforcement della coreografia

Nel contesto dell'enforcement a run-time introdotto nella Sezione 4.6.3, i Coordination Delegates sono i responsabili della distribuzione delle interazioni che avvengono tra i partecipanti della missione (e quindi della coreografia). Partendo dalla specifica globale, essi vengono sintetizzati in modo automatico e successivamente vengono inseriti tra i partecipanti con lo scopo di fargli controllare le comunicazioni e prevenire così comportamenti non ammessi dalla specifica stessa [1, 3].
L'approccio adottato (come per il caso di studio mostrato nel Capitolo 5) si basa sul coordinamento distribuito, infatti ogni delegate coopera con gli altri senza la necessità di un orchestratore centrale, aggiornando lo stato della coreografia tramite l'osservazione del flusso delle diverse comunicazioni tra i partecipanti [1, 3].
Nel momento in cui vengono inseriti tra due o più servizi, si occupano di coordinare lo scambio di informazioni di coordinamento (automaticamente generate dalla specifica) per consentire/bloccare le diverse interazioni [1].
Dal punto di vista operativo i Coordination Delegates implementano un algoritmo di mutua esclusione adattato al contesto coreografico che sfrutta le nozioni di *relazione happened-before*, *ordinamento parziale* e *time-stamp* [22] per la gestione dell'accesso alle interazioni. L'enforcement in questo modo avviene tramite la gestione coordinata di rami indipendenti e, in caso di conflitti tra interazioni, i delegates cooperano per determinare quale sequenza deve precedere le altre [3].

4.7.2 Coordination Delegates e partecipanti black-box

In una coreografia può accadere che i partecipanti della missione siano servizi di terze parti (ergo non modificabili), e quindi il problema del coordinamento assume una configurazione diversa, dato che il comportamento interno dei componenti non può essere in alcun modo modificato/alterato. In tal caso ottenere la logica di coordinamento distribuito per constatare la realizzabilità della coreografia è un compito non banale e soggetto ad errori, dal momento in cui anche le sequenze di interazione vengono ristrette, eliminandone quelle non conformi [3].
Poiché in questo contesto i partecipanti hanno il comportamento interno non modificabile, il loro linguaggio (che indichiamo con $L(P)$) resta sempre fisso. I Coordination Delegates non possono quindi né modificare questo linguaggio, né aggiungere nuove sequenze; il loro compito consiste solamente nell'eliminare quelle non ammesse [3]. Da tutto ciò ne consegue che il comportamento del sistema, per rispettare la specifica globale, deve essere sia un sottoinsieme di $L(P)$, sia di $L(C)$ (linguaggio della coreografia).

Questa condizione è espressa tramite la formula 4.10:

$$L_{enf} = L(P) \cap L(C) \tag{4.10}$$

dove l'enforcement (indicato con L_{enf})) risulta possibile soltanto nel caso in cui l'intersezione (che esprime la natura restrittiva dell'azione dei Coordination Delegates) non sia vuota ($\varnothing$). Se allo stesso tempo non esistono sequenze possibili per i partecipanti e ammesse dalla coreografia, non esiste nessuna strategia di coordinamento che renda il sistema conforme alla specifica [1].
Nel contesto dei MRS affrontato nel Capitolo 5 tale vincolo risulta particolarmente rilevante poiché i controller dei robot, essendo già implementati e non soggetti a modifica, rendono necessario un coordinamento che operi esclusivamente a livello di interazioni.

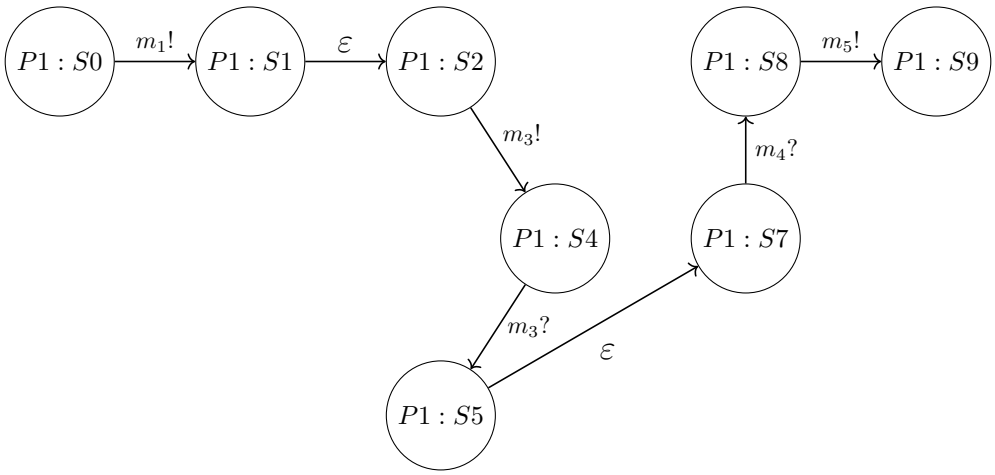
4.8 Le quattro fasi dell'approccio

Dopo aver discusso la nozione di coreografia, la realizzabilità ed i Coordination Delegates, la presente sezione conclude il capitolo descrivendo il processo che conduce alla sintesi del sistema coordinato.

4.8.1 Proiezione

La prima fase dell'approccio consiste nella proiezione della coreografia globale sui ruoli coinvolti nel sistema. Partendo dalla specifica globale, che viene descritta come modello che descrive le interazioni dei partecipanti, per ciascun componente si calcola una vista locale rappresentante il comportamento atteso all'interno della coreografia. La proiezione mantiene quindi esclusivamente le interazioni in cui il componente è direttamente coinvolto, rimuovendo quelle ritenuti irrilevanti per il ruolo preso in considerazione [3]. I partecipanti alla fine avranno un modello astratto ciascuno, che descrive le sequenze di interazioni che esso deve essere in grado di realizzare per rispettare la coreografia, ma non rappresenta i comportamenti concreti dei partecipanti. Tale modello sarà utilizzato nelle fasi successive dell'approccio per verificare se risulta compatibile con i comportamenti reali e per guidare i Coordination Delegates [3].
Questa parte si conclude presentando le proiezioni dei quattro partecipanti, ottenute a partire dalla specifica globale mostrata in Figura 4.2 nella Sezione 4.2.1. In particolare: le azioni di invio verranno indicate con l'etichetta **m!**, quelle di ricezione **m?** e le transizioni che non corrispondono ad un evento di comunicazione osservabile dal partecipante con $\boldsymbol{\varepsilon}$.
La Figura 4.6a mostra le proiezioni di P1 e P2; la 4.6b quelle di P3 e P4.

P1:



P2:

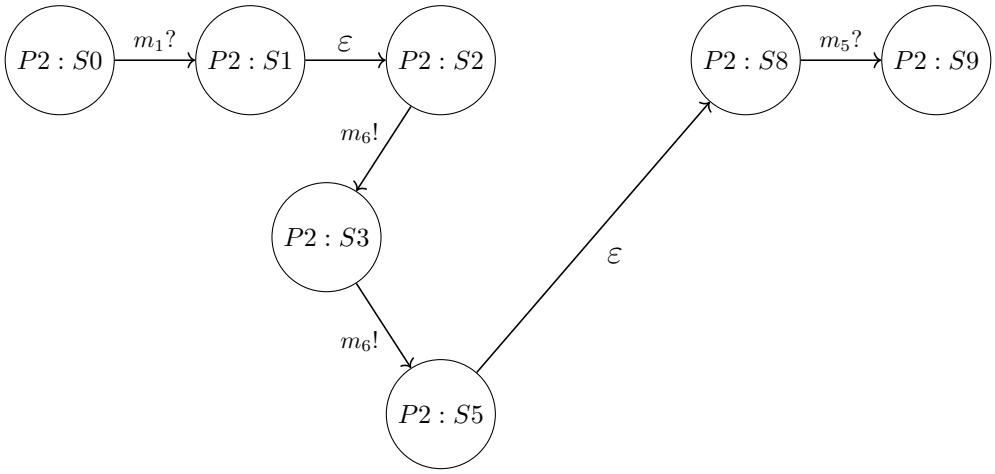


Figura 4.6a: Proiezione della coreografia sui partecipanti P_1 e P_2.

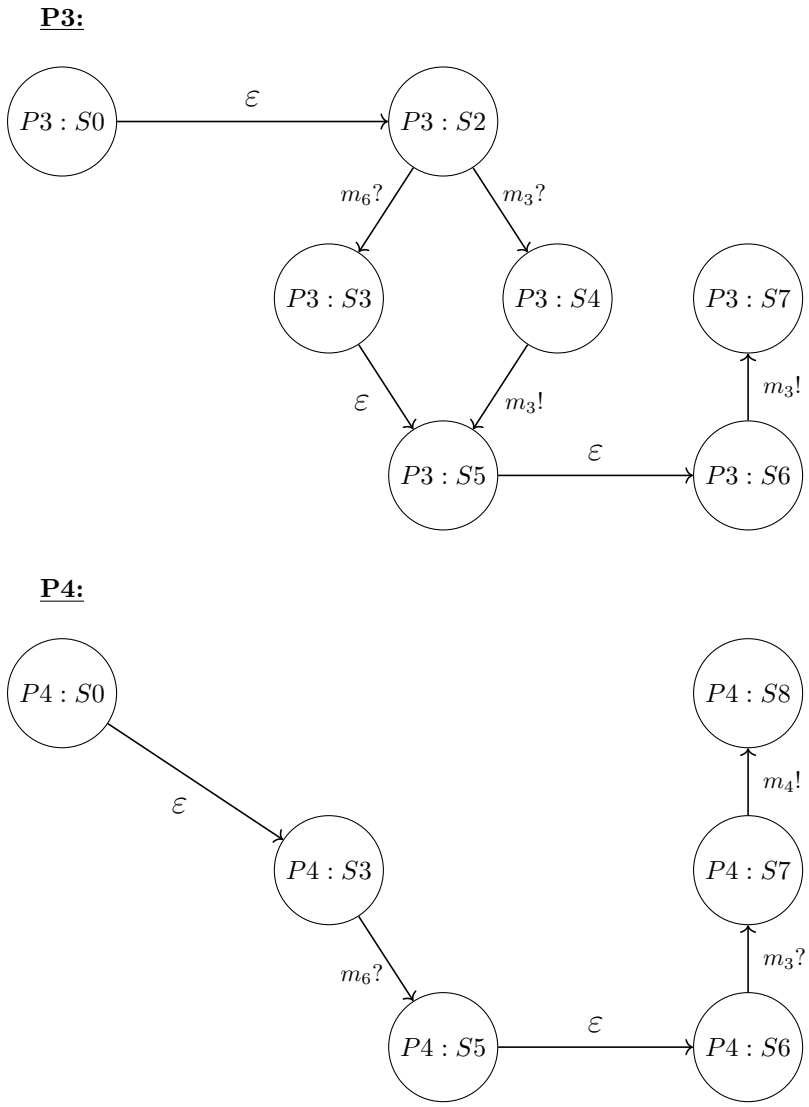


Figura 4.6b: Proiezione della coreografia sui partecipanti P_3 e P_4.

4.8.2 Selezione

Dopo la fase di proiezione l'approccio procede con la fase di selezione dei partecipanti che si occuperanno di roprire i ruoli individuati; in questa fase, la coreografia non viene implementata da zero, ma si basa sull'utilizzo di componenti già esistenti (anche sviluppati da terze parti) che vengono selezionati da un insieme di servizi disponibili e che devono essere in grado di svolgere il ruolo previsto dalla coreografia [3]. Ciascun partecipante presente all'interno di questo insieme ha una specifica comportamentale del proprio protocollo di interazione, modellata come una macchina a stati, dove [3]:

- Le transizioni rappresentano azioni di invio e ricezione messaggi.
- Sono ammesse anche azioni non osservabili da altri componenti.
- Sono presenti flussi paralleli, diramazioni e punti di sincronizzazione.

- Per ogni transizione è specificato se la comunicazione è sincrona, asincrona o interna.

La selezione dei partecipanti concreti avviene tramite la verifica della compatibilità comportamentale, formalizzata attraverso la relazione di raffinamento alternante, che viene introdotta per controllare in modo automatico se il comportamento di un partecipante concreto (ottenuto dall'insieme di servizi disponibili ed indicato con P_i^c) sia compatibile con quello astratto (P_i) ottenuto tramite la proiezione della coreografia [3]; questo passaggio garantisce che il partecipante concreto sia compatibile con il ruolo assegnato e che non introduca sequenze non ammissibili.
Nel caso in cui l'insieme dei servizi selezionati copra un piccolo sottoinsieme dei ruoli previsti dalla specifica globale, il comportamento astratto dei ruoli rimanenti verrà utilizzato per generare automaticamente il codice che andrà ad implementare da zero i partecipanti mancanti, che verranno pubblicati nell'inventario dei servizi disponibili per un riuso futuro [3].
Un aspetto fondamentale nella fase di selezione risiede nella distinzione tra ciò che fa il singolo servizio (business logic) e il modo in cui l'insieme di servizi viene coordinato (coordination logic) per garantire il rispetto della specifica globale. I servizi selezionati sono responsabili esclusivamente dell'esecuzione delle funzionalità di propria competenza, mentre il coordinamento viene generato automaticamente sotto forma di Coordination Delegates [3], senza richiedere una modifica al codice dei partecipanti concreti.

4.8.3 Sintesi e concretizzazione

L'approccio prosegue con la fase di sintesi, che prende in ingresso la specifica della coreografia e successivamente genera in modo automatico un insieme di modelli di comportamento astratto dei Coordination Delegates [3]. Questi ultimi sono modellati come macchine a stati, dove le azioni di invio, ricezione, ed interne sono rappresentate dalle transizioni.
Il modello dei CDs include anche transizioni relative ai messaggi di coordinamento e sincronizzazione, necessari per realizzare la specifica globale. Poiché il comportamento astratto dei delegates deriva dalla coreografia, esso non prende in considerazione le modalità di comunicazione dei partecipanti; tali aspetti vengono introdotti nell'ultima fase, quella di concretizzazione, che produce il comportamento concreto del Coordination Delegate (che utilizza code di messaggi vincolate ad uno per la gestione dell'asincronia [5, 4]). In questo contesto la comunicazione standard può essere sia sincrona che asincrona; la comunicazione di coordinamento invece è sempre sincrona.
Per l'insieme dei Coordination Delegates sintetizzati vale la proprietà di correttezza per costruzione, ovvero che una volta combinati con i partecipanti selezionati, il sistema poi realizza la coreografia specificata.

4.8.4 Confronto tra le quattro fasi dell'approccio

Alla luce di quanto descritto nelle Sezioni 4.8.1, 4.8.2 e 4.8.3, la Tabella 4.13 conclude questa parte mostrando un confronto tra le quattro fasi appena descritte [3].

Fase	Input	Output	Oggetto modellato	Obiettivo	Livello di astrazione
Proiezione	Coreografia	Abstract Participant Behavior	Partecipanti astratti	Derivare comportamenti locali	Astratto
Selezione	Service Inventory	Partecipanti concreti compatibili	Partecipanti concreti	Verifica di compatibilità (raffinamento)	Concreto (ma ancora senza coordinamento)
Sintesi	Coreografia + partecipanti selezionati	CD Abstract Behavior	Coordination Delegates (astratti)	Generare logica di coordinamento	Astratto
Concretizzazione	CD Abstract Behavior	CD Concrete Behavior	Coordination Delegates (concreti)	Gestione asincronia (code 1-bounded)	Concreto

Tabella 4.13: Confronto tra le quattro fasi dell'approccio.

Capitolo 5

Metodo automatico per la collaborazione tra MRS

5.1 Introduzione del metodo automatico

L'obiettivo del metodo che viene presentato in questa sezione è quello di ricostruire in modo automatico tutte le interazioni tra i robot che cooperano per svolgere una missione condivisa. In particolare, il metodo deve individuare correttamente tutte le interazioni collaborative tra i robot che sono necessarie per lo svolgimento della missione, per poi integrarle nei processi locali che descrivono il comportamento operativo di ciascun robot, partendo:

- Dai loro comportamenti basilari, ovvero i loro processi locali originali, privi di interazioni.
- Dal choreography diagram globale che descrive la missione complessiva.
- Da un'ontologia dei tipi di dato scambiati.

Il principio su cui si basa il metodo è il seguente: ogni interazione nasce nel momento in cui un robot produce un'informazione pubblica che un altro robot deve consumare, per proseguire il flusso operativo della missione. Il metodo automatico deve quindi:

1. Analizzare i task interni di ciascun robot e le informazioni prodotte.
2. Confrontare le informazioni prodotte con quelle richieste nei messaggi del choreography diagram globale.
3. Stabilire se esiste compatibilità (sintattica e semantica) tra i dati prodotti e quelli richiesti.
4. Determinare quali robot si devono attivare.
5. Determinare in che modo i robot si devono attivare.

6. Integrare automaticamente le interazioni di collaborazione trovate nei processi locali, nei punti coerenti con il flusso operativo di ogni robot.

Nel metodo automatico presentato in questa tesi, il caso $1 \rightarrow 1$ non è trattato separatamente, ma come un caso particolare del caso più generale $1 \rightarrow N$, dove N in questo caso = 1; questo evita di trattare separatamente i due casi.

5.2 Input del metodo automatico

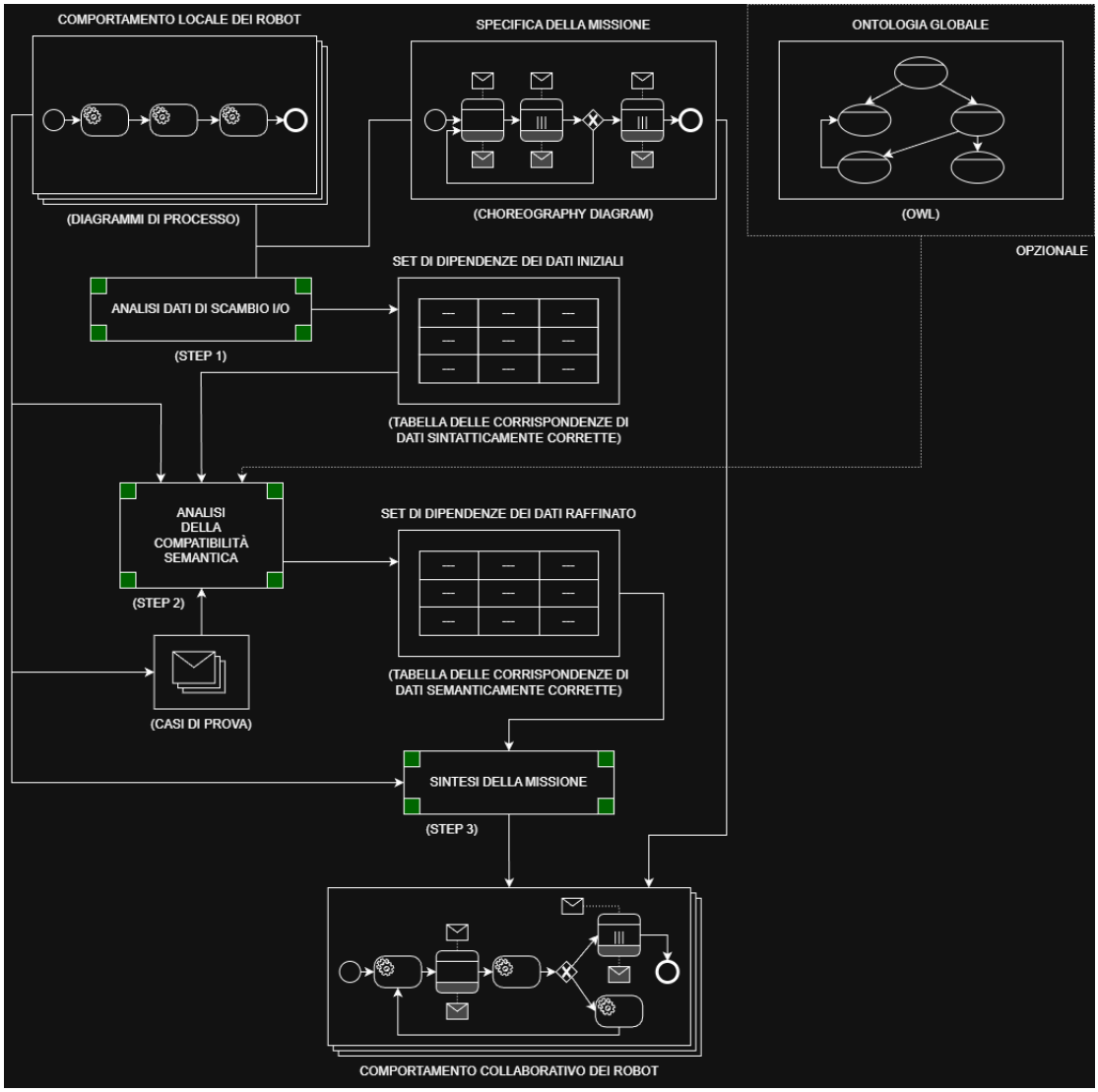


Figura 5.1: Diagramma esplicativo del metodo.

La Figura 5.1 fornisce una rappresentazione del metodo automatico proposto, evidenziando gli input, gli artefatti e l'output finale di tutto il processo di sintesi dell'intera collaborazione. Essa mostra come tutte le informazioni (comportamento locale dei robot, specifica globale della missione e semantica dei dati scambiati) vengano integrate

in modo progressivo dal metodo. Nella parte superiore della figura sono riportati tutti gli input del metodo.

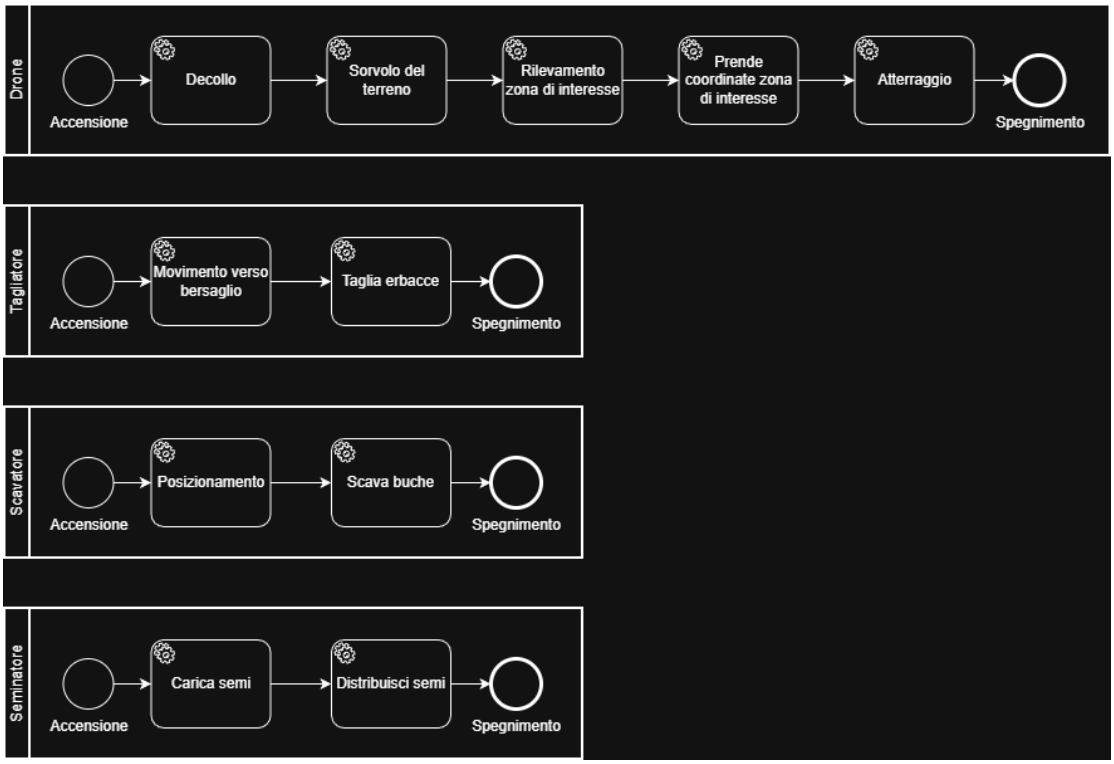


Figura 5.2: Operazioni basilari dei robot .

In particolare, i processi locali dei robot (Figura 5.2) descrivono le operazioni basilari eseguite in modo autonomo da ciascun robot, costituendo così il punto di partenza del nostro metodo.
Il choreography diagram globale fornisce invece la specifica astratta del coordinamento, e si occupa di definire tutte le interazioni necessarie per il raggiungimento dello scopo della missione.
L'ontologia globale dei tipi di dato (opzionale) può essere utilizzata per arricchire l'analisi di tutte le compatibilità tra i dati prodotti ed utilizzati dai robot, soprattutto in casi di corrispondenza multipla.
A partire da tali input, il metodo costruisce un insieme di dipendenze dei dati, basato inizialmente su criteri di compatibilità sintattica, successivamente però raffinato attraverso criteri di compatibilità semantica. Questi insiemi intermedi costituiscono artefatti abbastanza importanti per la determinazione di tutte le interazioni valide nel contesto dell'intera missione.
Il cuore del metodo è articolato in tre fasi computazionali, rappresentate da rettangoli con ai bordi dei piccoli quadrati verdi, ed evidenziate come STEP 1 (analisi dati di scambio Input/Output), STEP 2 (analisi della compatibilità semantica) e STEP 3 (sintesi dell'intera missione). Grazie a queste fasi, il metodo determina le interazioni che devono essere integrate, e in quali punti dei processi locali devono essere collocate.

Il risultato finale del metodo è la generazione di processi locali collaborativi, nei quali le operazioni basilari dei robot si integrano perfettamente con le attività di coreografia, producendo un coordinamento distribuito coerente con la specifica globale della missione.
La figura, quindi, evidenzia il ruolo del metodo come meccanismo che collega una descrizione astratta del coordinamento con la realizzazione operativa dei comportamenti locali.

5.3 Descrizione generale del metodo

Nella versione iniziale del metodo automatico, il processo di individuazione dei match tra i dati prodotti dai robot e i messaggi del choreography diagram globale veniva presentato come un legame $1 \to 1$, cioè ogni informazione prodotta da un robot risultava compatibile con un solo messaggio e, di conseguenza, con un solo destinatario. Se vogliamo estendere questo pensiero, possiamo considerare il caso $1 \to 1$ come una situazione particolare del caso generale $1 \to N$, dove N = 1. Il metodo, infatti, non deve distinguere i due casi: il match $1 \to 1$ è semplicemente un match $1 \to N$ in cui uno solo dei possibili destinatari risulta semanticamente compatibile.
Nel generalizzare il metodo, riconosciamo quindi che in un sistema multi-robot può accadere che la stessa informazione prodotta da un robot sia utile a più partecipanti, generando situazioni di uno a molti. Questa generalizzazione richiede che il metodo automatico sia in grado di:

1. Riconoscere la presenza di più match compatibili.
2. Comprendere quali robot devono effettivamente attivarsi.
3. Stabilire in che modo e in quale ordine devono farlo (sequenza, parallelismo o condizione esclusiva).

Il primo passo del metodo generalizzato consiste nel verificare se un'informazione prodotta da un robot può essere utile a più destinatari. Quando un robot esegue una delle sue attività interne, quest'ultima può generare un'informazione, come ad esempio le coordinate di una zona, un segnale di completamento operazione o una conferma. Il metodo automatico confronta il tipo di informazione prodotta con il tipo di output del contenuto dei messaggi presenti nel choreography diagram globale.

- Se esiste un solo messaggio che utilizza quel tipo di informazione, ci troviamo nel caso particolare di match $1 \to 1$ (ossia $1 \to N$ con N = 1).
- Se invece più messaggi trasportano lo stesso tipo di dato, il metodo riconosce che la stessa informazione può essere inviata a più robot, e quindi ci troviamo nel caso generale $1 \to N$.

In questa fase il metodo non decide ancora a chi inviare l'informazione, ma segnala la possibilità che essa possa essere destinata a più partecipanti, affidando la scelta finale alla fase di compatibilità semantica.

5.4 Compatibilità sintattica e semantica

Nei casi di match tra dati prodotti e dati richiesti, sia in quelli uno ad uno ($1 \rightarrow 1$) che in quelli multipli ($1 \rightarrow N$) non è sufficiente considerare la sola compatibilità sintattica tra i tipi di dato, poiché due informazioni possono avere la stessa forma ma significati diversi. Per questo motivo, il metodo automatico adotta una nozione di compatibilità semantica, che consente di distinguere tra semplici corrispondenze di formato e vere corrispondenze di significato. In particolare [7]:

- Parliamo di compatibilità sintattica quando due elementi condividono lo stesso tipo o struttura di dato (per esempio, entrambi sono di tipo *Boolean* o *Coordinate*). In questa tesi distinguiamo due forme di compatibilità sintattica:
  - **Compatibilità sintattica "strong"**, dove i tipi devono coincidere esattamente. In questo caso il match è ammesso solo se il dato inviato e quello atteso dal robot destinatario condividono lo stesso tipo base senza variazioni.
  - **Compatibilità sintattica "weak"**, basata sulla relazione di sottotipo. Qui un dato è considerato compatibile anche quando il suo tipo è un sottotipo del tipo richiesto, cioè, contiene almeno tutte le informazioni necessarie al destinatario (anche con campi aggiuntivi). In questo caso, infatti, messaggi più "ricchi" possono comunque essere utilizzati da robot che richiedono solo un sottoinsieme dell'informazione.
- Si parla invece di compatibilità semantica quando due elementi non solo hanno lo stesso formato, ma rappresentano la stessa informazione logica o lo stesso concetto operativo all'interno dei processi dei robot.

Ad esempio, un dato **Coordinate** prodotto dal Drone è semanticamente compatibile con l'input **CoordinateZona** del Tagliatore solo se entrambi descrivono la posizione fisica della zona di lavoro, e non semplicemente una coppia numerica generica. In questo senso la compatibilità semantica si fonda su tre livelli ispirati a concetti presenti nell'ambito dell'interoperabilità e dello schema matching [25, 26]:

1. **Livello sintattico**: il tipo di dato coincide o è derivato dallo stesso tipo base.
2. **Livello strutturale**: la composizione interna del dato (campi e/o attributi) è equivalente.
3. **Livello semantico**: il significato operativo dell'informazione è coerente nel contesto della missione.

Il metodo automatico, dopo aver individuato più possibili match, utilizza queste tre verifiche per filtrare i candidati:

- Prima verifica la compatibilità sintattica, escludendo i messaggi di tipo diverso.
- Poi valuta la compatibilità semantica, controllando che il ruolo logico dell'informazione sia coerente con quello atteso dal robot ricevente.
- Infine, genera le interazioni solo se entrambi i criteri risultano soddisfatti.

Grazie a questa estensione, il metodo è in grado di evitare match non compatibili, riconoscendo come valide solo le interazioni che hanno un reale significato operativo condiviso. Il concetto di compatibilità semantica adottato in questo lavoro si ispira alla nozione introdotta nel framework Strawberry [7], nonché all'approccio proposto nel lavoro sui mediatori [2], dove la compatibilità tra servizi non è stabilita solo dai tipi di dato (compatibilità sintattica), ma può essere anche verificata tramite due ulteriori meccanismi:

1. Verifica comportamentale (testing) per accertare che la dipendenza sia semanticamente valida.
2. Utilizzo di concetti semantici e relazioni di sottotipo (subsumption) tra i tipi, tipiche delle ontologie.

Questi due meccanismi costituiscono una delle modalità tramite cui anche il metodo automatico proposto in questa tesi può determinare la compatibilità semantica nei casi $1 \rightarrow N$ in modo corretto, sebbene non completo.

5.5 Ontologie e subsumption

Un metodo fondamentale per verificare la compatibilità semantica tra informazioni consiste nell'utilizzare un'ontologia dei concetti informativi scambiati tra i robot. Un'ontologia è una rappresentazione formale che descrive [21]:

- I tipi di informazione del dominio.
- Le relazioni gerarchiche tra essi.
- Il significato operativo che questi tipi assumono all'interno dei processi.

Nel contesto del MRS considerato in questa tesi, i tipi di dato come **Coordinate**, **CoordinateZona**, **MapData**, **CoordinateTrattoreVicino** ecc. possono essere modellati come concetti dell'ontologia.
La relazione più importante è la subsumption (relazione di sottotipo semantico), che consente di stabilire quando un concetto è più generale o più specifico di un altro [2] (ad esempio CoordinateZona $\sqsubseteq$ Coordinate).
La subsumption è utile in due modi:

1. **Compatibilità sintattica weak**: se un tipo A è un sottotipo di B (A $\sqsubseteq$ B), allora l'informazione contenuta in A è sufficiente per soddisfare i requisiti di B. Vale a dire che il robot che utilizza B, può utilizzare anche A.

2. **Compatibilità semantica vera**: la relazione A $\sqsubseteq$ B garantisce anche che i due concetti sono semanticamente corretti, perché entrambi descrivono la stessa entità del dominio, ma con livello di dettaglio differenti.

Per esempio, il Drone produce un dato **datoA** e il Tagliatore richiede un input **datoB**. Se nell'ontologia viene definita una relazione di subsumption tra i due tipi di dato, come mostrato nella Figura 5.3:

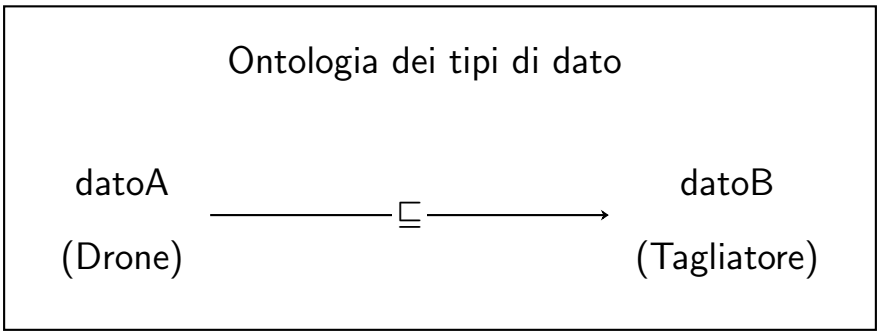


Figura 5.3: Riferimento ontologico.

Allora l'informazione del Drone è semanticamente compatibile con quella richiesta dal Tagliatore, anche se i nomi non coincidono sintatticamente. Grazie a questo modello, il metodo automatico può:

- Interpretare correttamente tipi di dato differenti ma semanticamente legati.
- Evitare match errati (come, per esempio, stringhe che hanno formati uguali ma significati diversi).
- Decidere quali robot devono ricevere un'informazione in caso di match multipli ($1 \rightarrow N$) basandosi sulla coerenza semantica, non solamente sulla forma del dato.

Questa strategia è ispirata agli approcci che sono stati proposti nella sintesi automatica di mediatori [2], dove le relazioni semantiche tra tipi sono validate tramite concetti e gerarchie formali, evitando match sintatticamente corretti, ma semanticamente inconsistenti.

5.6 Testing comportamentale

Un'ulteriore strategia, complementare all'uso delle ontologie, consiste nella verifica tramite testing comportamentale, ripreso dal framework Strawberry [7]. L'idea è semplice: un match sintattico tra due messaggi è semanticamente valido se e solo se, inserendo

l'informazione del mittente nel processo del destinatario, il comportamento risultante è coerente con quello atteso. Si tratta di effettuare un test simulato sul modello dei processi, verificando se l'informazione trasmessa:

- Può essere consumata dal task ricevente.
- Non genera violazioni o situazioni non ammissibili.
- Rispetta le precondizioni e gli effetti dei successivi task.
- Mantiene la coerenza di tutta la coreografia.

Facendo un esempio nel dominio preso in considerazione in questa tesi: se il Drone produce una mappa dell'area **MapData** e il metodo automatico trova due possibili match (Tagliatore e Scavatore), il testing comportamentale simula due cose:

1. Cosa succede se la mappa viene utilizzata dal Tagliatore.
2. Cosa succede se la mappa viene utilizzata dallo Scavatore.

Il sistema quindi verifica se:

- Il processo del robot può iniziare correttamente (quindi le precondizioni sono state soddisfatte).
- Il task successivo può essere eseguito.
- L'informazione ha senso in quel contesto.

Se l'informazione permette al processo del Tagliatore di procedere correttamente (ma non lo fa con quello dello Scavatore), il metodo conclude che:

- Il match $Drone \rightarrow Tagliatore$ è semanticamente corretto.
- Di conseguenza il match $Drone \rightarrow Scavatore$ non lo è.

Il testing di questo tipo permette di risolvere un problema nei match $1 \rightarrow N$, ovvero che i tipi uguali corrispondono ad un match sintattico, ma in robot con ruoli diversi solo alcuni match sono semanticamente validi. Quindi con questo tipo di approccio, vengono scartati i match "falsi positivi" (tipo uguale ma significato diverso) e si mantengono solo quelli che rispettano il comportamento previsto.

5.7 Analisi e disambiguazione dei possibili destinatari

Dopo aver trovato più possibili corrispondenze, il metodo deve capire quali robot sono effettivamente interessati allo scambio di informazioni e in che modo devono reagire. Per farlo, viene adottato un processo di analisi suddiviso in vari passaggi:

1. **Analisi semantica dei tipi di dato**: il metodo confronta la natura dell'informazione che è stata prodotta con ciò che ciascun robot si aspetta di ricevere. Solo i robot che possiedono un task con un input compatibile con quel tipo di dato vengono considerati destinatari calidi. Questo filtro elimina ovviamente match semanticamente errati.

2. **Analisi delle dipendenze operative**: per i robot che hanno passato il primo punto, il metodo adesso osserva la posizione delle attività nei processi locali dei robot (vale a dire l'ordine in cui normalmente vengono svolte) per capire se le loro azioni devono avvenire in sequenza o possono avvenire in qualsiasi ordine, anche in parallelo. Se un task ricevente logicamente si trova dopo un altro robot nel flusso, il metodo lo interpreta come sequenziale; se le azioni invece non hanno dipendenze dirette, il metodo lo interpreta come match *interleaved*, anche detto, per semplicità, *parallelo*.

3. **Analisi della sincronizzazione**: in alcuni casi solo uno tra più robot deve reagire, ad esempio quello più vicino o disponibile. In queste situazioni, il metodo interpreta la comunicazione come condizionata, ovvero che viene gestita da un meccanismo di scelta. Nella Sezione 5.11.1 viene descritto un esempio di questo genere utilizzato nel caso di studio presentato in questo lavoro di tesi.

Una volta chiarito quali robot devono reagire e in che modo, il metodo costruisce automaticamente la parte di choreography diagram corrispondente. Quindi, in base al tipo di relazione individuata, se:

- L'interazione coinvolge un solo destinatario ⇒ viene creato un semplice choreography task singolo (caso $1 \rightarrow 1$).

- L'informazione deve raggiungere più robot contemporaneamente ⇒ viene inserito un gateway parallelo divergente, con un ramo in uscita per ogni destinatario (caso $1 \rightarrow N$ parallelo).

- I robot devono agire in un ordine specifico ⇒ vengono generati più choreography task collegati sequenzialmente (caso $1 \rightarrow N$ sequenziale).

- Solo uno dei robot deve intervenire ⇒ viene aggiunto un gateway esclusivo che seleziona il destinatario in base alla condizione logica (caso $1 \rightarrow N$ condizionato).

In questo modo il metodo è in grado di rappresentare automaticamente anche comportamenti di coordinamento più complessi, mantenendo la coerenza del choreography diagram. In sostanza, quando il metodo automatico trova più possibili corrispondenze, capisce chi deve muoversi e in che modo combinando le seguenti informazioni:

- Il contenuto semantico del dato (in questo modo chi non ne ha bisogno viene escluso).

- La posizione delle attività nei processi locali (per capire se le azioni devono essere in sequenza, in parallelo o sequenziali).
- Le possibili condizioni logiche o criteri di scelta (che servono ad attivare il robot corretto).

In questo modo viene permesso al metodo di generare automaticamente una coreografia coerente, dove ogni robot si attiva al momento giusto.

5.8 Applicazione del caso di studio

In questa sezione vengono analizzate le interazioni che consentono ai quattro robot di collaborare e di portare a termine la missione.
È necessario quindi fare riferimento alle operazioni basilari di ciascun robot (Figura 5.2, Sezione 5.2), che costituiscono il punto di partenza per costruire le interazioni collaborative.

5.8.1 Interazioni che partono dal Drone

5.8.1.1 Prima interazione

Il task "Prende coordinate zona di interesse" produce il dato **posizione_zona: Coordinate**, che rappresenta la posizione geografica della zona con le erbacce da rimuovere. Il metodo automatico deve confrontare i tipi di dati:

- L'output che produce il drone dopo il rilevamento sono numeri di Coordinate.
- Il messaggio "weedPositionMessage" rappresenta il contenitore dell'informazione prodotta dal Drone durante il rilevamento. Questo messaggio trasporta un dato di tipo **Coordinate**, avente lo stesso formato del dato prodotto localmente (*posizione_zona*).

Il match è stato trovato e viene generato/posizionato correttamente il choreography task.

5.8.1.2 Seconda interazione

Dopo la raccolta di tutte le disponibilità, il gateway "Tagliatore disponibile?" rappresenta la valutazione che consente al Drone di individuare il tagliatore più vicino alla zona di interesse. Il risultato di questa valutazione corrisponde a un'informazione interna, rappresentabile con il seguente dato, che verrà successivamente utilizzato per il confronto dal metodo automatico:

- **trattore_vicino: CoordinateTrattoreVicino**, che rappresenta le coordinate del trattore selezionato per il taglio. Il dato include la posizione geografica del trattore più vicino e un identificativo univoco (ID intero) che lo distingue dagli altri tagliatori.

Il messaggio "closestTractorMessage" rappresenta il contenitore dell'informazione prodotta dal Drone durante la comunicazione col trattore più vicino. Questo messaggio trasporta un dato di tipo CoordinateTrattoreVicino, avente anch'esso lo stesso formato del dato prodotto localmente (*trattore_vicino*).
Il match è stato trovato e viene generato/posizionato correttamente il choreography task.

5.8.2 Interazioni che partono dal Tagliatore

5.8.2.1 Prima interazione

Dopo l'avvio del processo locale del Tagliatore, il robot effettua un'autovalutazione del proprio stato operativo per determinare se è disponibile o no a partecipare alla missione. Il risultato di questa valutazione interna corrisponde ad un'informazione logica rappresentabile con il seguente dato (che verrà successivamente utilizzato dal metodo automatico per il confronto):

- **disponibilità: Boolean**, che assume valore true se il Tagliatore è libero, oppure false se è attualmente occupato o non pronto all'interazione.

Il messaggio "AvailabilityMessage" rappresenta il contenitore dell'informazione prodotta dal Tagliatore durante la comunicazione con il Drone. Questo messaggio trasporta un dato di tipo **Boolean**, avente lo stesso formato del dato prodotto localmente (*disponibilità*).
Il match è stato trovato e viene generato/posizionato correttamente il choreography task.

5.8.2.2 Seconda interazione

Dopo aver completato il taglio delle erbacce, il Tagliatore genera un'informazione sullo stato del campo, rappresentando il fatto che la superficie è pronta per la fase successiva di scavo. Questa informazione corrisponde al seguente dato interno:

- **campo_pronto: Boolean**, che indica se il terreno è stato completamente liberato da erbacce e reso idoneo per lo scavatore, con il valore true in caso positivo, false altrimenti.

Il messaggio "fieldAlreadyMessage" rappresenta il contenitore dell'informazione prodotta dal Tagliatore durante la comunicazione con lo Scavatore. Questo messaggio trasporta

un tipo di dato **Boolean**, avente lo stesso formato e la stessa semantica del dato prodotto localmente (*campo_pronto*).
Il match è stato trovato e viene generato/posizionato correttamente il choreography task.

5.8.3 Interazioni che partono dallo Scavatore

5.8.3.1 Prima interazione

Una volta terminata la fase di scavo, lo Scavatore produce un'informazione interna che descrive lo stato di completamento dell'operazione. Tale informazione è rappresentabile attraverso il dato:

- **scavo_completato: Boolean**, che assume valore true se la preparazione del terreno è conclusa con successo, false altrimenti.

Il messaggio "digDoneMessage" rappresenta il contenitore dell'informazione prodotta dallo Scavatore durante la comunicazione con il Seminatore. Questo messaggio trasporta un dato di tipo **Boolean**, avente lo stesso formato del dato che è stato prodotto in locale (*scavo_completato*).
Il match è stato trovato e viene generato/posizionato correttamente il choreography task.

5.8.4 Interazioni che partono dal Seminatore

5.8.4.1 Prima interazione

Dopo aver terminato la distribuzione dei semi, il Seminatore produce un'informazione sullo stato complessivo della missione, che certifica l'avvenuto completamento di tutte le operazioni. Tale informazione è rappresentata dal seguente dato:

- **missione_completata: Boolean**, che assume valore true quando la semina è terminata correttamente ed il robot è pronto a concludere la missione.

Il messaggio "seedingDoneMessage" rappresenta il contenitore dell'informazione prodotta dal Seminatore e trasmessa in broadcast verso il sistema di monitoraggio o verso gli altri robot. Questo messaggio trasporta un dato di tipo **Boolean**, avente lo stesso formato del dato prodotto localmente (*missione_completata*).
Il metodo, quindi, individua il match ed inserisce l'ultimo choreography task nel processo del Seminatore, notificando la conclusione coordinata della missione all'intero sistema.

5.8.5 Esempio esteso $1 \rightarrow N$

Nel caso di studio originale, ogni interazione tra robot è di tipo $1 \rightarrow 1$. Il Drone comunica con il Tagliatore, il Tagliatore con lo Scavatore, e lo Scavatore con il Seminatore.

Tuttavia, per illustrare il funzionamento del metodo automatico in scenari più complessi, si può considerare un'estensione in cui la stessa informazione prodotta da un robot viene condivisa con più partecipanti contemporaneamente. Possiamo immaginare che, dopo aver individuato la zona di interesse (l'area dove sono presenti le erbacce da tagliare), il Drone non comunichi questa informazione solo al Tagliatore, ma anche allo Scavatore e al Seminatore, non per farli agire in modo immediato, ma per permettere loro di preparare in anticipo la pianificazione delle proprie attività future. In questa fase quindi:

- Il Tagliatore userà le coordinate per decidere il percorso di taglio delle erbacce.
- Lo Scavatore le utilizzerà per capire dove posizionarsi una volta ricevuto il segnale di "terreno pronto".
- Il Seminatore potrà stimare in anticipo l'area su cui dovrà intervenire.

Tutti e tre i robot, quindi, ricevono la stessa informazione (le coordinate della zona di interesse), ma la utilizzano in modi e momenti diversi del loro processo operativo.
Il task interno del Drone "Prende coordinate zona di interesse" produce un'informazione di tipo: **zona_interesse: MapData**, rappresentante la mappa dell'area da elaborare. Questa informazione prodotta può essere associata a più messaggi nel choreography diagram, che vedremo nella Tabella 5.1:

Messaggio	Destinatario	Informazione trasportata
weedPositionMessage	Tagliatore	MapData
digPlanMessage	Scavatore	MapData
seedPlanMessage	Seminatore	MapData

Tabella 5.1: Esempio di messaggi scambiati nel caso di studio esteso $1 \rightarrow N$.

Il metodo automatico, analizzando i tipi di dato, individua che tutti e tre i messaggi trasportano informazioni di tipo **MapData**, e quindi viene riconosciuto un match $1 \rightarrow N$ tra Drone, Tagliatore, Scavatore e Seminatore.
A questo punto, il metodo automatico applica la propria logica di analisi, descritta nella Sezione 5.7:

1. **Analisi semantica**: conferma che tutti i robot destinatari hanno processi che richiedono una mappa dell'area per la pianificazione (input compatibile con MapData).
2. **Analisi delle dipendenze operative**: rileva che nessuno dei tre robot necessita dell'esecuzione dell'altro per poter ricevere la mappa (quindi l'attivazione può avvenire in parallelo).

3. **Analisi della sincronizzazione**: non sono presenti condizioni che potrebbero condurre ad esclusione; quindi, l'informazione viene trasmessa nello stesso momento a tutti.

Il metodo automatico classifica quindi il match come $1 \rightarrow N$ parallelo e genera nel choreography diagram la seguente struttura:

- Dopo il task interno del Drone "Prende coordinate zona di interesse", viene inserito un gateway parallelo divergente.
- Dal gateway partono tre choreography task:
  1. "Invia coordinate al Tagliatore" tramite il messaggio "weedPositionMessage".
  2. "Invia coordinate allo Scavatore" tramite il messaggio "digPlanMessage".
  3. "Invia coordinate al Seminatore" tramite il messaggio "seedPlanMessage".
- Ognuno di questi task è collegato al rispettivo destinatario, che riceve il dato **MapData** ed aggiorna la propria pianificazione locale.

La conclusione è: dal punto di vista logico, il Drone si comporta da **pubblicatore**, distribuendo la stessa informazione a più destinatari che la consumano in modo indipendente. I robot non agiscono ancora fisicamente, ma aggiornano le proprie "mappe di lavoro" in modo coerente.
Questo esempio dimostra che il metodo automatico, anche senza modificare la logica dell'orto reale, è in grado di:

- Riconoscere la presenza di più match compatibili.
- Classificare il tipo di caso.
- Generare, come nel caso $1 \rightarrow 1$, tutte le interazioni per far sì che il caso di studio funzioni correttamente.

5.9 Legenda dei diagrammi BPMN

Dopo aver descritto il funzionamento del metodo automatico proposto, nelle sezioni seguenti verranno presentati tutti i diagrammi utilizzati per la rappresentazione del caso di studio. Essi consentono di visualizzare sia le interazioni tra i robot, sia il loro coordinamento.
Iniziamo con la Figura 5.4, che riporta la legenda dei simboli che sono stati utilizzati nei diagrammi BPMN che verranno presentati nel seguito del capitolo, alcuni dei quali già descritti in 4.2.2.3. Tale legenda ha lo scopo di chiarire il significato semantico di ciascun elemento grafico [23], garantendo che i diagrammi siano interpretati in modo

non ambiguo e coerente, indipendentemente dal tipo di diagramma o dal contesto in cui vengono utilizzati.



Figura 5.4: Legenda dei diagrammi BPMN.

In particolare, verranno adottate le seguenti convenzioni:

- **Inizio evento**: rappresentato da un cerchio rosso e delle volte da un cerchio vuoto con bordo sottile, indica l'avvio del processo o del comportamento del robot. Nei diagrammi di proiezione e negli enhanced diagrams, l'inizio dell'evento sostituisce esplicitamente i task di accensione, rappresentando l'inizio logico dell'esecuzione del robot.

- **Fine evento**: rappresentato da un cerchio verde e delle volte da un cerchio vuoto con bordo più spesso di quello di Inizio evento, indica la conclusione del processo o della missione del robot. Analogamente ad Inizio evento, Fine evento sostituisce i task di spegnimento, segnando così la terminazione del comportamento di ciascun robot.

- **Attività di coreografia (Choreography task)**: descrive un'interazione tra due o più partecipanti ed è rappresentata da un rettangolo suddiviso in tre parti; nella parte superiore ed inferiore vanno rispettivamente mittente e destinatario, nella parte centrale invece viene visualizzato il messaggio. Queste attività rappresentano il cuore del coordinamento distribuito tra i robot.

- **Attività di servizio (Service task)**: rappresentata da un rettangolo con icona a ingranaggio, indica un'operazione interna al robot, non osservabile dall'esterno e non direttamente coinvolta nello scambio di messaggi.

- **Messaggio iniziale**: rappresentato da un'icona a forma di busta, che indica l'invio di un messaggio da un partecipante ad un altro, avviando così una comunicazione asincrona.

- **Messaggio di ritorno**: rappresentato anch'esso tramite un'icona a busta, leggermente più scura, che indica la risposta o la conferma a un messaggio precedentemente ricevuto.

- **Gateway esclusivo divergente**: anche chiamato XOR, viene utilizzato per modellare una scelta condizionale e consente l'attivazione di uno e un solo ramo tra quelli disponibili, in base ad una condizione che si verifica.

- **Gateway parallelo convergente**: anche chiamato AND, viene utilizzato per sincronizzare più flussi di esecuzione, attendendo il completamento di tutte le attività parallele prima di proseguire.

- **Evento start timer**: rappresentato da un cerchio con icona di orologio, indica l'attivazione del processo o di un ramo di esecuzione a seguito del verificarsi di una condizione temporale, come un time-out oppure un'attesa prefissata.

Questa legenda è valida per tutti i diagrammi che verranno presentati nel seguito del capitolo, inclusi il choreography diagram globale, i diagrammi di projection, quelli enhanced, e costituisce un riferimento comune per l'interpretazione dei modelli.

5.10 Choreography diagram globale

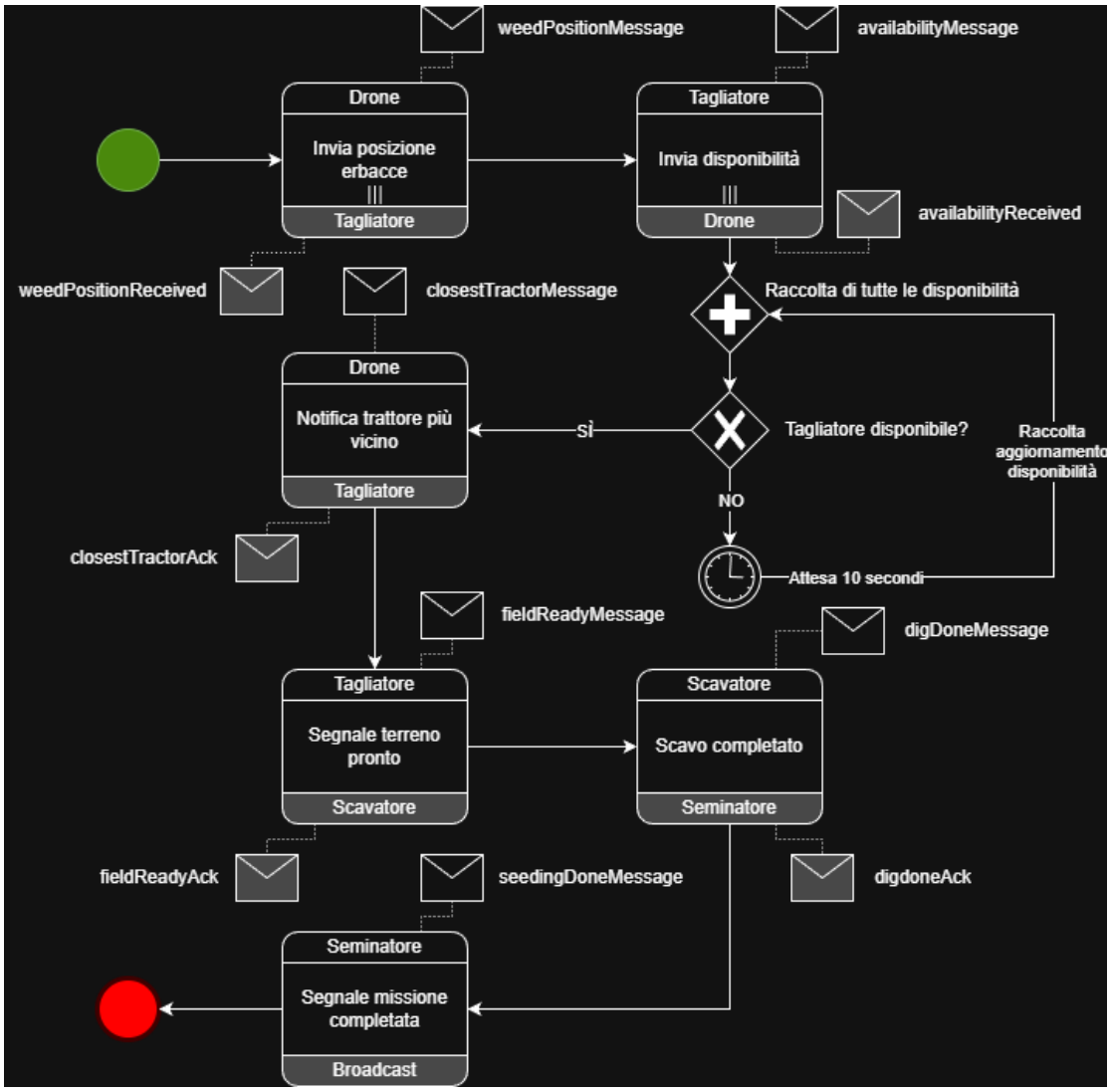


Figura 5.5: Choreography diagram globale.

La Figura 5.5 rappresenta la vista complessiva del coordinamento tra i robot coinvolti nella missione, descrivendo le interazioni ammesse tra i partecipanti e l'ordine logico con cui tali interazioni possono avvenire. A differenza dei diagrammi di processo locali questo diagramma non modella il comportamento interno dei singoli robot, ma si concentra esclusivamente sullo scambio di messaggi e sulle dipendenze informative che regolano la collaborazione distribuita.

Nel diagramma sono presenti quattro partecipanti principali: Drone, Tagliatore, Scavatore e Seminatore. Ciascuno di questi è coinvolto in una o più attività di coreografia che definiscono le fasi fondamentali della missione collaborativa. Il flusso di interazioni inizia quando il Drone invia al Tagliatore la posizione delle erbacce tramite il messaggio "weedPositionMessage". A seguito di questa prima comunicazione, il Tagliatore risponde comunicando la proprià disponibilità attraverso il messaggio "availabilityMessage". A

questo punto il diagramma presenta una fase di raccolta delle disponibilità, designata da un gateway parallelo convergente. Tale costrutto indica che il sistema deve sincronizzare (ovvero raccogliere) tutte le informazioni di disponibilità previste prima di procedere con il passo successivo. Conclusa quindi la raccolta, è presente un gateway esclusivo divergente che verifica la condizione "Tagliatore disponibile?":

- Se la risposta è "sì", allora il Drone notifica al Tagliatore quale sia il trattore più vicino tramite il messaggio "closestTractorMessage".
- Se la risposta è "no", il processo entra in un ramo di attesa, dove inizialmente è presente un evento timer di attesa (10 secondi). Al termine di questa attesa, il flusso esegue una fase di aggiornamento delle disponibilità e rientra nel gateway AND, realizzando quindi un ciclo temporizzato: il sistema ripete in modo periodico la verifica finché la disponibilità non risulta soddisfatta.

Quando la disponibilità è confermata e la selezione del trattore è stata notificata, il Tagliatore segnala allo Scavatore che il terreno è pronto tramite "fieldReadyMessage". Successivamente lo Scavatore comunica al Seminatore il completamento dello scavo tramite "digDoneMessage". Infine, il Seminatore conclude il flusso segnalando il completamento della missione tramite "seedingDoneMessage" con destinatario Broadcast, indicando una notifica finale rivolta a tutti i partecipanti della missione.

5.11 Diagrammi di proiezione

I diagrammi di proiezione rappresentano la vista locale del comportamento di ciascuno dei quattro robot, e sono ottenuti proiettando il choreography diagram globale su un singolo partecipante. Ciascun diagramma descrive come il robot preso in considerazione percepisce e gestisce le interazioni collaborative, mantenendo esclusivamente le attività in cui esso è direttamente coinvolto.
Questa struttura è quindi rappresentata tramite una cooperazione distribuita basata su dipendenze piuttosto che su un controllo con un robot che agisce da orchestratore. In tali diagrammi, il significato dei choreography task, dei gateway e degli eventi temporali è lo stesso già descritto per il choreography diagram globale, pertanto non verrà ulteriormente ripetuto.
L'obiettivo della proiezione non è ridefinire il comportamento complessivo del sistema, ma evidenziare il punto di vista locale di ogni singolo robot, e il modo in cui esso si inserisce e collabora nel coordinamento distribuito.
In questi diagrammi compare il simbolo "?" associato ad alcuni partecipanti nei choreography task; tale notazione non riguarda mai il robot su cui è costruita la proiezione, ma l'entità remota con cui esso interagisce. Il simbolo "?" indica quindi un interlocutore che, nella fase di proiezione, non è ancora univocamente determinabile, oppure non è ancora rilevante per la definizione del comportamento locale.

La risoluzione dei partecipanti concreti avverrà in una fase successiva del metodo automatico proposto, nella quale le interazioni vengono integrate nei processi locali, assegnando ruoli specifici sulla base di criteri di compatibilità e selezione.

5.11.1 Proiezione del Drone

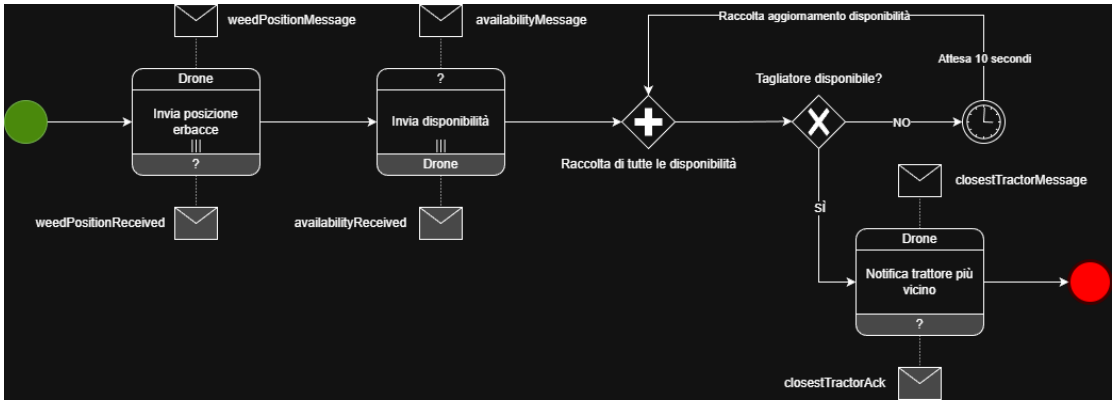


Figura 5.6: Diagramma di projection del Drone.

La Figura 5.6 mette in evidenza il ruolo del robot come attivatore e coordinatore iniziale della missione collaborativa.
Dal punto di vista locale, il Drone avvia il flusso di interazione e gestisce le informazioni di disponibilità necessarie alla selezione del robot a cui assegnare il compito successivo. Il comportamento del Drone include una fase di sincronizzazione delle disponibilità ricevute e una successiva decisione condizionata che determina la prosecuzione del processo. In assenza di condizioni favorevoli, il Drone adotta un comportamento di attesa temporizzata e reiterazione del controllo, mentre, in caso positivo, procede con la notifica dell'assegnazione al robot selezionato.

5.11.2 Proiezione del Tagliatore

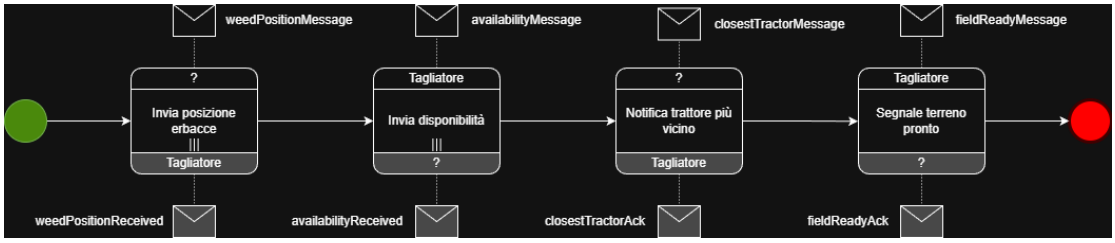


Figura 5.7: Diagramma di projection del Tagliatore.

La Figura 5.7 rappresenta il comportamento locale del robot incaricato della preparazione del terreno.
Dal punto di vista del Tagliatore, il flusso di interazione ha inizio con la ricezione delle informazioni relative alla posizione delle erbacce; successivamente, il Tagliatore comunica

la propria disponibilità al sistema e rimane in attesa di una possibile futura assegnazione. Quando riceve la notifica di selezione, il Tagliatore procede con l'esecuzione delle attività operative previste e, al termine, segnala che il terreno è pronto per la fase successiva della missione.

5.11.3 Proiezione dello Scavatore

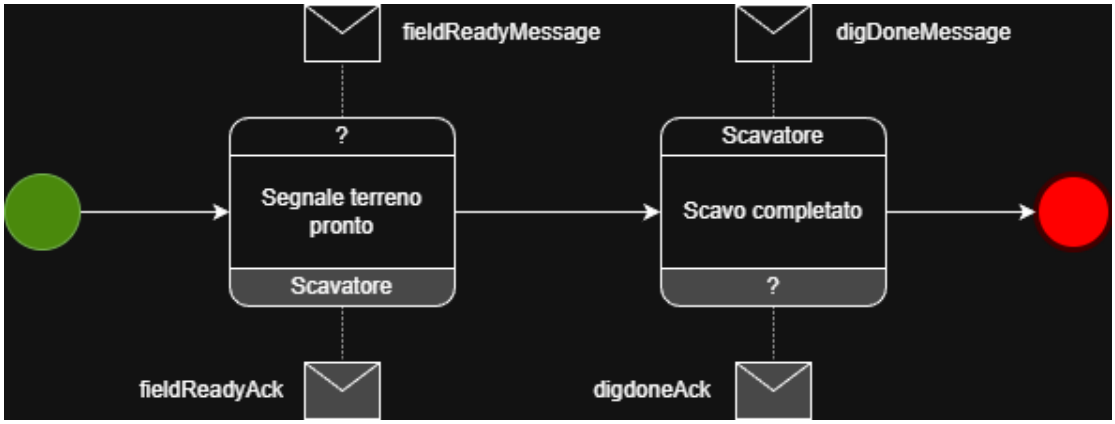


Figura 5.8: Diagramma di projection dello Scavatore.

La Figura 5.8 descrive il comportamento locale del robot incaricato dell'esecuzione della fase di scavo.
Qui il flusso di interazione si attiva solo dopo la ricezione del segnale che indica la preparazione del terreno. Dopo aver ricevuto tale informazione, il passo successivo comprende quello di notificare il completamento dello scavo all'ultimo robot che costituisce questa missione, il Seminatore.

5.11.4 Proiezione del Seminatore

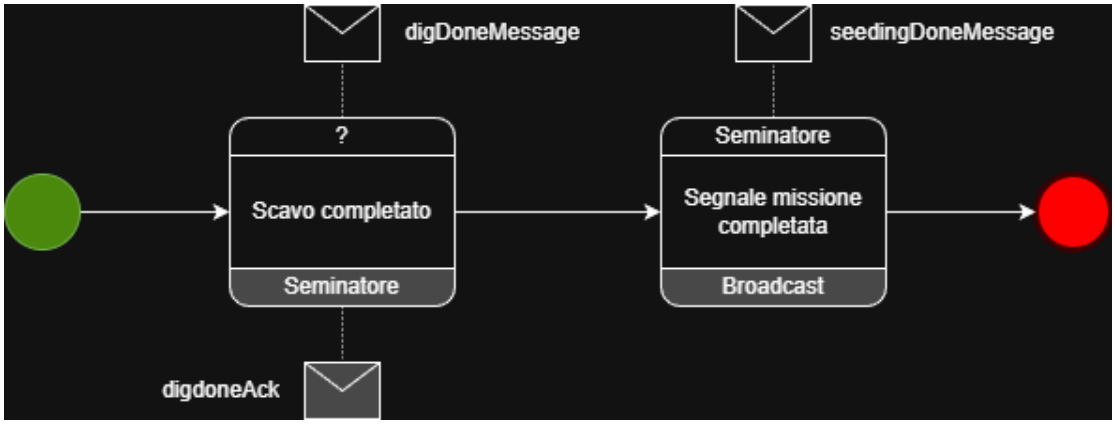


Figura 5.9: Diagramma di projection del Seminatore.

La Figura 5.9 descrive il comportamento locale del robot incaricato per la fase conclusiva della missione di coordinamento distribuito.

Dal punto di vista del Seminatore, il flusso di interazione viene attivato a seguito della ricezione della notifica di completamento dello scavo. Una volta ricevuta tale informazione, il Seminatore esegue le attività di propria competenza e conclude il proprio contributo segnalando il completamento della missione.

5.12 Diagrammi enhanced

I diagrammi migliorati, o diagrammi enhanced, rappresentano l'evoluzione dei diagrammi di projection ottenuta applicando il metodo automatico proposto. L'obiettivo è quello di integrare le interazioni collaborative all'interno dei processi operativi locali di ciascun robot.
In questa fase, le attività di coreografia vengono collegate nei punti appropriati del flusso di esecuzione, insieme a tutte le operazioni basilari che ogni singolo robot detiene. In questo modo, vengono descritti tutti i processi locali completi.

5.12.1 Drone enhanced

La Figura 5.10 mostra il diagramma migliorato del Drone.

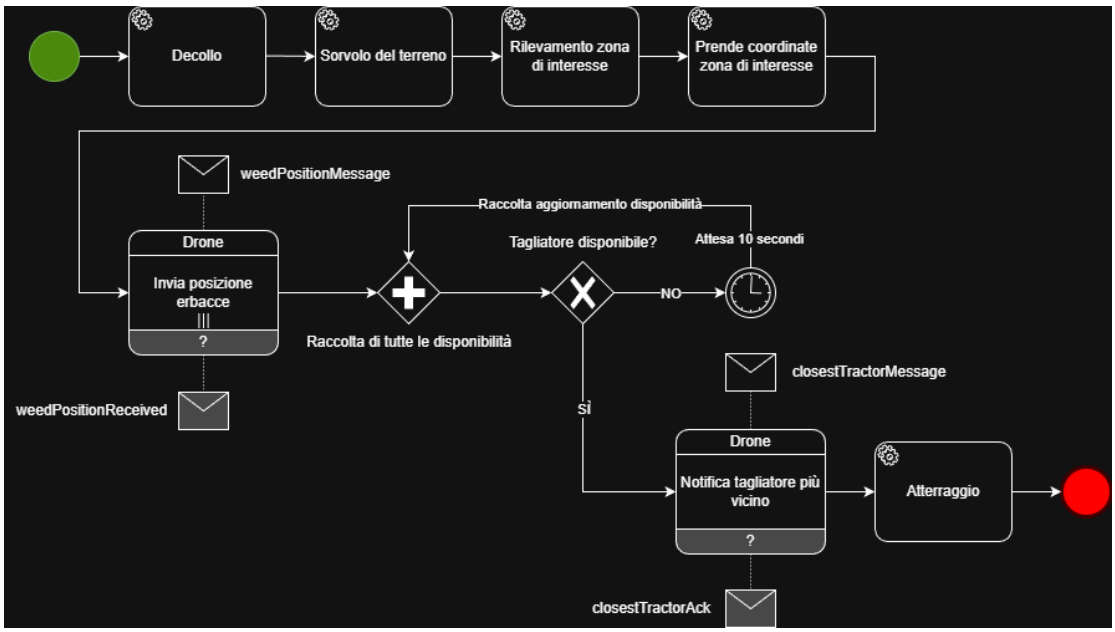


Figura 5.10: Diagramma enhanced del Drone.

5.12.2 Tagliatore enhanced

La Figura 5.11 riporta il diagramma migliorato del Tagliatore.

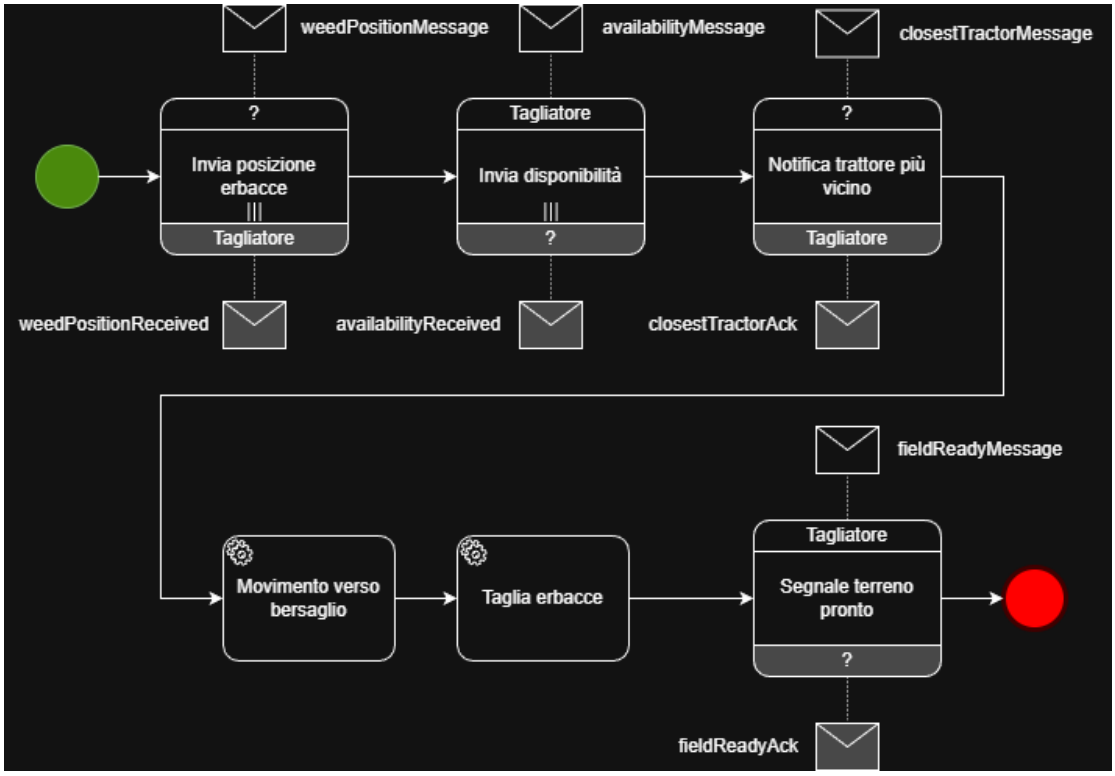


Figura 5.11: Diagramma enhanced del Tagliatore.

5.12.3 Scavatore enhanced

Il diagramma migliorato dello Scavatore è riportato in Figura 5.12.

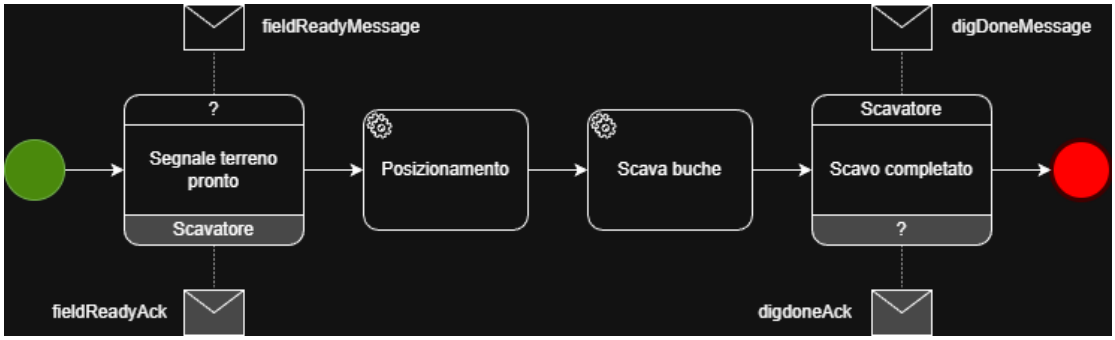


Figura 5.12: Diagramma enhanced dello Scavatore.

5.12.4 Seminatore enhanced

La Figura 5.13 rappresenta il diagramma migliorato del Seminatore.

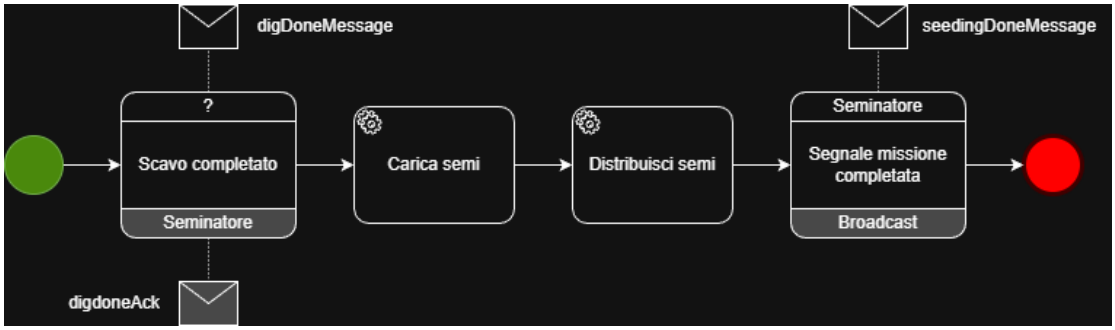


Figura 5.13: Diagramma enhanced del Seminatore.

5.13 Unified collaboration diagram

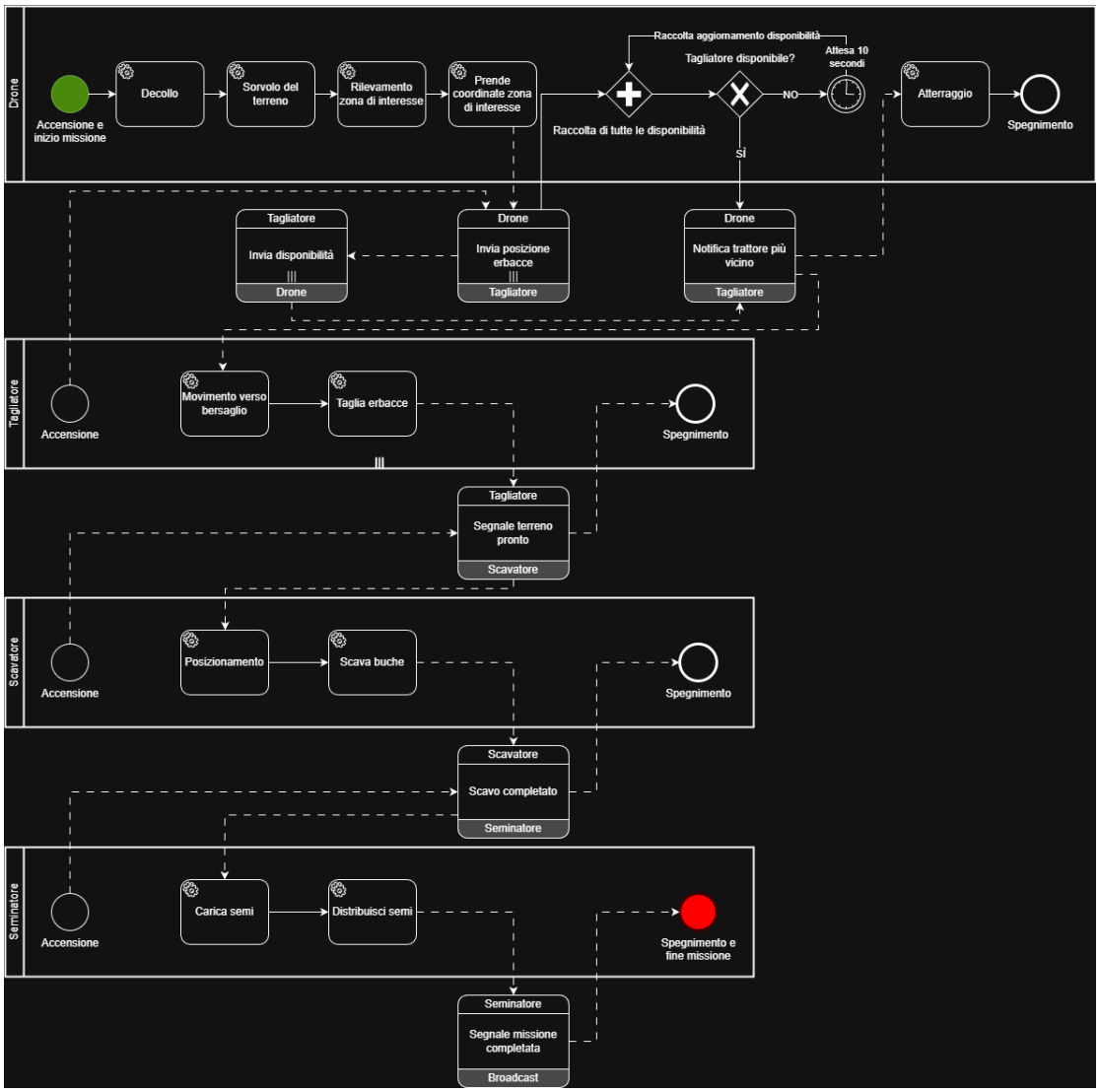


Figura 5.14: Unified collaboration diagram.

La Figura 5.14 fornisce una vista complessiva del sistema multi-robot ottenuta integrando i diagrammi enhanced dei singoli robot. Tale rappresentazione riunisce in un unico

modello i comportamenti locali, le operazioni basilari e tutte le interazioni di collaborazione.
Lo scopo principale di questo diagramma è quello di mostrare come le operazioni basilari di ciascun robot e i choreography task risultino correttamente sincronizzati, ricorrendo a dipendenze e scambi di messaggi, senza fare utilizzo di un controllo centralizzato. Esso costituisce quindi una verifica della correttezza e coerenza del coordinamento distribuito in un sistema multi-robot, ottenuto tramite il metodo automatico proposto in questo lavoro di tesi.

Capitolo 6

Conclusioni e sviluppi futuri

Nel corso di questo lavoro, oltre ad aver analizzato i MRS e i problemi relativi al coordinamento distribuito, è stato approfondito il ruolo delle coreografie nella descrizione delle interazioni ed evidenziato le criticità legate alla realizzabilità delle specifiche globali.
Successivamente è stato analizzato il ruolo dei meccanismi di enforcement come soluzione per garantire il rispetto della specifica globale senza l'introduzione di un orchestratore centrale. I Coordination Delegates in questo contesto permettono la distribuzione della logica di coordinamento tra componenti, poiché osservano e coordinano tutte le comunicazioni tra i partecipanti, scartando le interazioni che non risultano conformi alla coreografia.
L'applicazione ad un caso di studio dell'approccio proposto ha evidenziato il modo in cui i robot possono essere coordinati in modo coerente attraverso la gestione delle dipendenze tra operazioni e la sincronizzazione delle interazioni.
Il lavoro presentato in questo elaborato fornisce due possibili sviluppi futuri: estendere l'approccio proposto a diversi modelli con lo scopo di analizzarne l'applicabilità in contesti differenti, e sviluppare strumenti software in grado di supportare ed automatizzare le diverse fasi di processo, partendo dalla definizione della specifica globale fino alla generazione dei meccanismi di coordinamento distribuito.

Ringraziamenti

A conclusione di questo elaborato, desidero menzionare tutte le persone, senza le quali questo lavoro di tesi non esisterebbe nemmeno.

Ringrazio il mio relatore Prof. Massimo Tivoli, che in questi mesi di lavoro è stato sempre presente e disponibile, guidandomi con suggerimenti nelle ricerche e nella stesura dell'elaborato.

Un pensiero speciale va a mia nonna Annunziata, che non è qui a vedere questo traguardo, ma che continua ad accompagnarmi in ogni passo.

Un infinito ringraziamento va ai miei genitori, Antonella e Angelo, che con il loro sostegno, la loro presenza e i sacrifici fatti nel corso degli anni mi hanno permesso di intraprendere e portare a termine questo percorso universitario. A loro devo molto di questo traguardo.

Un ringraziamento speciale va a Noemi, che per me non è soltanto una sorella ma anche la mia migliore amica. Grazie per essermi sempre stata accanto, per il tuo sostegno, per la fiducia e per l'affetto che non mi hai mai fatto mancare. Ti voglio bene.

Ringrazio i miei nonni, Berardina e Berardino, per l'affetto, il sostegno e per essere sempre stati orgogliosi di me. Grazie per non avermi mai fatto mancare nulla e per la vostra presenza costante.

Ringrazio tutti i miei zii e cugini per l'affetto che mi dimostrano ogni giorno. Grazie per il vostro sostegno e per il bene che mi avete sempre fatto sentire.

Ringrazio infinitamente Andrea, che in questo percorso mi ha sempre aiutato e si è fatto sempre trovare pronto nel momento del bisogno.

Ringrazio Federico e Paolo, miei fratelli non di sangue, ma per scelta. Il primo, per questioni lavorative non è sempre presente fisicamente, ma la sua vicinanza non manca mai, sia tramite messaggio, sia quando torna in città.

Per ultimi (ma non per importanza) ringrazio Beatrice, Dario, Giorgia, Ionela, Lorenzo e Stefano che, oltre ad aver condiviso con me questo percorso, sono stati una presenza fondamentale anche al di fuori dell'ambito universitario tra uscite, risate, confronti e momenti di leggerezza.

Riferimenti

[1] Marco Autili, Davide Di Ruscio, Amleto Di Salle, Paola Inverardi, and Massimo Tivoli. A model-based synthesis process for choreography realizability enforcement. In *International Conference on Fundamental Approaches to Software Engineering*, pages 37–52. Springer, 2013.

[2] Marco Autili, Paola Inverardi, Romina Spalazzese, Massimo Tivoli, and Filippo Mignosi. Automated synthesis of application-layer connectors from automata-based specifications. *Journal of Computer and System Sciences*, 104:17–40, 2019.

[3] Marco Autili, Paola Inverardi, and Massimo Tivoli. Choreography realizability enforcement through the automatic synthesis of distributed coordination delegates. *Science of Computer Programming*, 160:3–29, 2018.

[4] Samik Basu and Tevfik Bultan. Automated choreography repair. In *International Conference on Fundamental Approaches to Software Engineering*, pages 13–30. Springer, 2016.

[5] Samik Basu, Tevfik Bultan, and Meriem Ouederni. Deciding choreography realizability. *Acm Sigplan Notices*, 47(1):191–202, 2012.

[6] Mordechai Ben-Ari and Francesco Mondada. *Elements of robotics*. Springer Nature, 2017.

[7] Antonia Bertolino, Paola Inverardi, Patrizio Pelliccione, and Massimo Tivoli. Automatic synthesis of behavior protocols for composable web-services. In *Proceedings of the 7th joint meeting of the European software engineering conference and the ACM SIGSOFT symposium on The foundations of software engineering*, pages 141–150, 2009.

[8] Standard Bots. Multi-robot systems: What they are and how they work. `https://standardbots.com/blog/multi-robot-systems-what-they-are-how-they-work`, 2025.

[9] Olaf Burkart, Didier Caucal, Faron Moller, and Bernhard Steffen. Chapter 9 - verification on infinite structures. In J.A. Bergstra, A. Ponse, and S.A. Smolka,

editors, *Handbook of Process Algebra*, pages 545–623. Elsevier Science, Amsterdam, 2001.

[10] Nadia Busi, Roberto Gorrieri, Claudio Guidi, Roberto Lucchi, and Gianluigi Zavattaro. Towards a formal framework for choreography. In *14th IEEE International Workshops on Enabling Technologies: Infrastructure for Collaborative Enterprise (WETICE'05)*, pages 107–112. IEEE, 2005.

[11] Marco Carbone and Fabrizio Montesi. Deadlock-freedom-by-design: multiparty asynchronous global programming. *ACM SIGPLAN Notices*, 48(1):263–274, 2013.

[12] Flavio Corradini, Andrea Morichetta, Andrea Polini, Barbara Re, and Francesco Tiezzi. Collaboration vs. choreography conformance in bpmn. *Logical Methods in Computer Science*, 16, 2020.

[13] Flavio Corradini, Sara Pettinari, Barbara Re, Lorenzo Rossi, and Francesco Tiezzi. A bpmn-driven framework for multi-robot system development. *Robotics and Autonomous Systems*, 160:104322, 2023.

[14] Edson de Araújo Silva, Eduardo Valentin, Jose Reginaldo Hughes Carvalho, and Raimundo da Silva Barreto. A survey of model driven engineering in robotics. *Journal of Computer Languages*, 62:101021, 2021.

[15] Gero Decker, Oliver Kopp, and Alistair Barros. An introduction to service choreographies (servicechoreographien–eine einführung). *it-Information Technology*, 50(2):122–127, 2008.

[16] Gero Decker, Johannes Maria Zaha, and Marlon Dumas. Execution semantics for service choreographies. In *International Workshop on Web Services and Formal Methods*, pages 163–177. Springer, 2006.

[17] Davide Di Ruscio, Ivano Malavolta, Patrizio Pelliccione, et al. A family of domain-specific languages for specifying civilian missions of multi-robot systems. In *MORSE@ STAF*, pages 16–29, 2014.

[18] M Bernardine Dias, Robert Zlot, Nidhi Kalra, and Anthony Stentz. Market-based multirobot coordination: A survey and analysis. *Proceedings of the IEEE*, 94(7):1257–1270, 2006.

[19] Alessandro Farinelli, Luca Iocchi, and Daniele Nardi. Multirobot systems: a classification focused on coordination. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, 34(5):2015–2028, 2004.

[20] Brian P Gerkey and Maja J Matarić. A formal analysis and taxonomy of task allocation in multi-robot systems. *The International journal of robotics research*, 23(9):939–954, 2004.

[21] Nicola Guarino, Daniel Oberle, and Steffen Staab. What is an ontology? In *Handbook on ontologies*, pages 1–17. Springer, 2009.

[22] Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Commun. ACM*, 21(7):558–565, July 1978.

[23] Business Process Model. Notation (bpmn) version 2.0. *OMG Specification, Object Management Group*, 19:52–60, 2011.

[24] Lynne Parker. *Multiple Mobile Robot Systems*, pages 921–941. Springer, 01 2008.

[25] Stanislav Pokraev, Dick Quartel, Maarten WA Steen, and Manfred Reichert. Semantic service modeling: Enabling system interoperability. In *Enterprise Interoperability: New Challenges and Approaches*, pages 221–230. Springer, 2007.

[26] Erhard Rahm and Philip A Bernstein. A survey of approaches to automatic schema matching. *the VLDB Journal*, 10(4):334–350, 2001.

[27] Zhi Yan, Nicolas Jouandeau, and Arab Ali Cherif. A survey and analysis of multi-robot coordination. *International Journal of Advanced Robotic Systems*, 10(12):399, 2013.